Security Review Report
NM-0411-0491 Rollup Boost

NETHERMIND SECURITY

(June 9, 2025)

Contents

1 Executive Summary 2

2 Audited Files 3

3 Summary of Issues 3

4 Summary of Security Implications to Permissioneless Builders 3

5 System Overview 4
- 5.1 Rollup Boost: Proposer-Builder Separation in L2 . . . 4
- 5.2 Core System Workflow of Rollup Boost . . . 4
- 5.3 HTTP Proxy Layer and Multiplexing Logic in `proxy.rs` . . . 5
- 5.4 Main Service Logic: The Role of `server.rs` . . . 5
- 5.5 RPC Module (`rpc.rs`) . . . 6

6 Risk Rating Methodology 8

7 Issues 9
- 7.1 [Medium] JWT unencrypted in HTTP connections . . . 9
- 7.2 [Medium] Server listening on all interfaces by default, without authentication . . . 9
- 7.3 [Low] Debug API authentication . . . 9
- 7.4 [Low] JWT tokens timestamped 60 seconds into the future . . . 9
- 7.5 [Low] Possible execution_mode inconsistency in `get_payload` . . . 10
- 7.6 [Low] Proxy does not implement rate-limiting . . . 10
- 7.7 [Low] Use of `sync::Cache` in an async context . . . 10
- 7.8 [Info] CI GitHub Action actions-rs unmaintained . . . 11
- 7.9 [Info] Code inside the `is_fallback_enabled()` branch may not be reachable . . . 12
- 7.10 [Info] Incoming requests have a large size limit . . . 13
- 7.11 [Info] `boost_sync` flag can cause discrepancies between builder and L2 op-geth . . . 14
- 7.12 [Info] getPayload requests to builder when FCU `no_tx_pool` is true . . . 15
- 7.13 [Best Practice] Concurrency in server . . . 15
- 7.14 [Best Practice] Different conditions for cache insertion/eviction . . . 15
- 7.15 [Best Practice] Do not clone payload in server|fcu_v3 . . . 16
- 7.16 [Best Practice] Silent failures of `tokio::spawn` . . . 16
- 7.17 [Best Practice] Using expect and unwrap in the code . . . 16
- 7.18 [Best Practice] CI use cargo deny . . . 16
- 7.19 [Best Practice] Do not clone payload in rpc|new_payload_v3 . . . 17
- 7.20 [Best Practice] Logic of forwarding requests should be improved . . . 17

8 Security Implications of Using Permissioneless Builders 18
- 8.1 MEV extraction attack like the one performed against Mev-Boost . . . 18
- 8.2 Do not blindly default to builder block . . . 18
- 8.3 Slow down (or even disable) *op-geth* with extremely large block from builder . . . 18

9 Documentation Evaluation 19
- 9.1 The code for `get_payload` is not easy to reason about . . . 19

10 Test Suite Evaluation 22
- 10.1 Code Coverage Analysis . . . 22
- 10.2 Tests Output . . . 23

11 About Nethermind 24

1 Executive Summary

This document presents the security review performed by Nethermind Security for Rollup Boost. Rollup Boost is a specialized sidecar service designed to enhance the flexibility and modularity of block production for Optimism Stack chains. It enables external block builders to construct blocks independently of the Sequencer's Execution Engine, allowing for more customizable transaction sequencing without modifying the core Optimism protocol.

The audited code comprises 2,634 lines of code written in Rust language. **The audit was performed using** (a) manual analysis of the codebase, (b) automated analysis tools, and (c) creation of test cases. Fig. 1 summarizes the the identified issues into four severity levels. **In this report, we present** 20 points of attention: two are classified as `Medium`, five are classified as `Low`, and 13 are classified as `Informational` or `Best Practice`. In addition, we also discuss **security implications** to permissioneless builders in Section 4.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 summarizes the issues and Section 4 summarizes security implications in the context of permissioneless builders. Section 5 presents the system overview. Section 6 discusses the risk rating methodology. Section 7 details the issues and Section 8 discuss the security implications of using permissioneless builders. Section 8 discusses the documentation provided by the client for this audit. Section 9 presents the compilation, tests, and automated tests. Section 10 concludes the document.

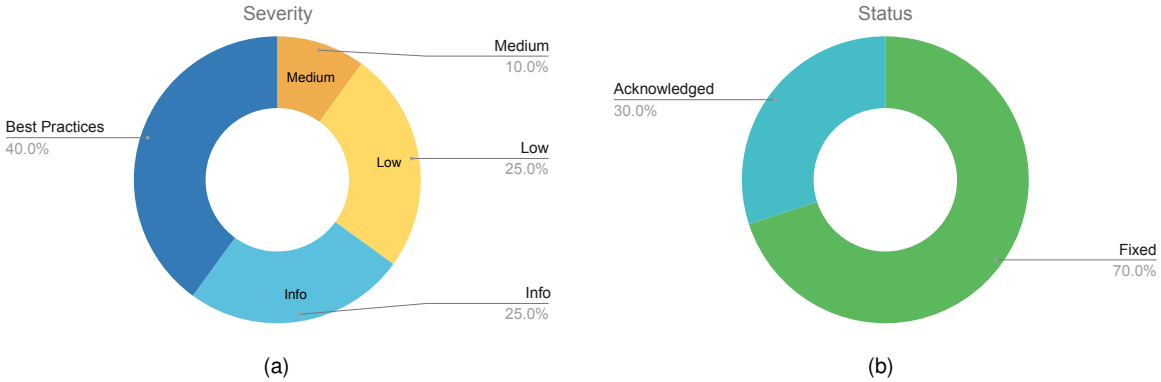

Fig. 1: Distribution of issues: Critical (0), **High** (0), **Medium** (2), **Low** (5), **Undetermined** (0), **Informational** (5), **Best Practices** (8). **Distribution of status: Fixed** (14), **Acknowledged** (6), **Mitigated** (0), **Unresolved** (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	May 11, 2025
Response from Client	Regular responses during audit engagement
Final Report	June 09, 2025
Repository	rollup-boost
Commit Audit	ea0ec43
Final Commit	71c4482
Documentation	README file and docs folder
Documentation Assessment	High
Test Suite Assessment	Medium

2 Audited Files

	Contract	LoC	Comments	Ratio	Blank	Total
1	src/server.rs	886	33	3.72%	122	11
2	src/proxy.rs	605	26	4.30%	108	708
3	src/client/rpc.rs	403	19	12.5%	50	128
4	src/cli.rs	176	22	6.77%	47	144
5	src/tracing.rs	148	10	7.32%	20	167
6	src/debug_api.rs	123	9	1.20%	36	34
7	src/client/http.rs	83	1	10.80%	15	10
8	src/client/auth.rs	65	7	1.70%	10	79
9	src/metrics.rs	59	1	1.70%	10	20
9	src/health.rs	57	1	1.75%	9	20
9	src/lib.rs	18	0	0.0%	8	20
9	src/bin/main.rs	8	0	0.0%	2	20
9	src/client/mod.rs	3	0	0.0%	0	20
	Total	**2,634**	**129**	**4.21%**	**437**	**1301**

3 Summary of Issues

	Finding	Severity	Update
1	JWT unencrypted in HTTP connections	Medium	Acknowledged
2	Server listening on all interfaces by default, without authentication	Medium	Fixed
3	Debug API authentication	Low	Acknowledged
4	JWT tokens timestamped 60 seconds into the future	Low	Fixed
5	Possible execution_mode inconsistency in `get_payload`	Low	Fixed
6	Proxy does not implement rate-limiting	Low	Acknowledged
7	Use of `sync::Cache` in an async context	Low	Fixed
8	CI GitHub Action actions-rs unmaintained	Info	Fixed
9	Code inside the `is_fallback_enabled()` branch may not be reachable	Info	Fixed
10	Incoming requests have a large size limit	Info	Fixed
11	`boost_sync` flag can cause discrepancies between builder and L2 op-geth	Info	Fixed
12	getPayload requests to builder when FCU `no_tx_pool` is true	Info	Fixed
13	Concurrency in server	Best Practices	Acknowledged
14	Different conditions for cache insertion/eviction	Best Practices	Fixed
15	Do not clone payload in server|fcu_v3	Best Practices	Fixed
16	Silent failures of `tokio::spawn`	Best Practices	Acknowledged
17	Using expect and unwrap in the code	Best Practices	Fixed
18	CI use cargo deny	Best Practices	Fixed
19	Do not clone payload in rpc|new_payload_v3	Best Practices	Fixed
20	Logic of forwarding requests should be improved	Best Practices	Acknowledged

4 Summary of Security Implications to Permissioneless Builders

	Description
1	Do not blindly default to builder block
2	Slow down (or even disable) op-geth with extremely large block from builder
3	MEV extraction attack like the one performed against Mev-Boost

5 System Overview

This section provides an overview of Rollup Boost at the initial audited commit `ea0ec43`, highlighting its architecture, components, and functionalities. **It is important to note that significant changes have occurred in the core logic during the resolution of identified issues. The final commit `71c4482` reflects these modifications, presenting the current state of the system post-review.**

5.1 Rollup Boost: Proposer-Builder Separation in L2

Ethereum Layer 1 has adopted a model called **Proposer-Builder Separation (PBS)**, where the block proposer delegates construction to an external builder that can optimize transaction ordering and MEV strategies. This separation, facilitated by tools such as `mev-boost`, enables modular and competitive block production.

Rollup Boost brings this PBS model to Layer 2 by decoupling the sequencer's execution logic from block construction, allowing external builders to generate blocks while maintaining protocol compatibility. This modularity opens up opportunities for experimentation, performance improvements, and greater decentralization in the OP Stack ecosystem. More information on the design here.

5.2 Core System Workflow of Rollup Boost

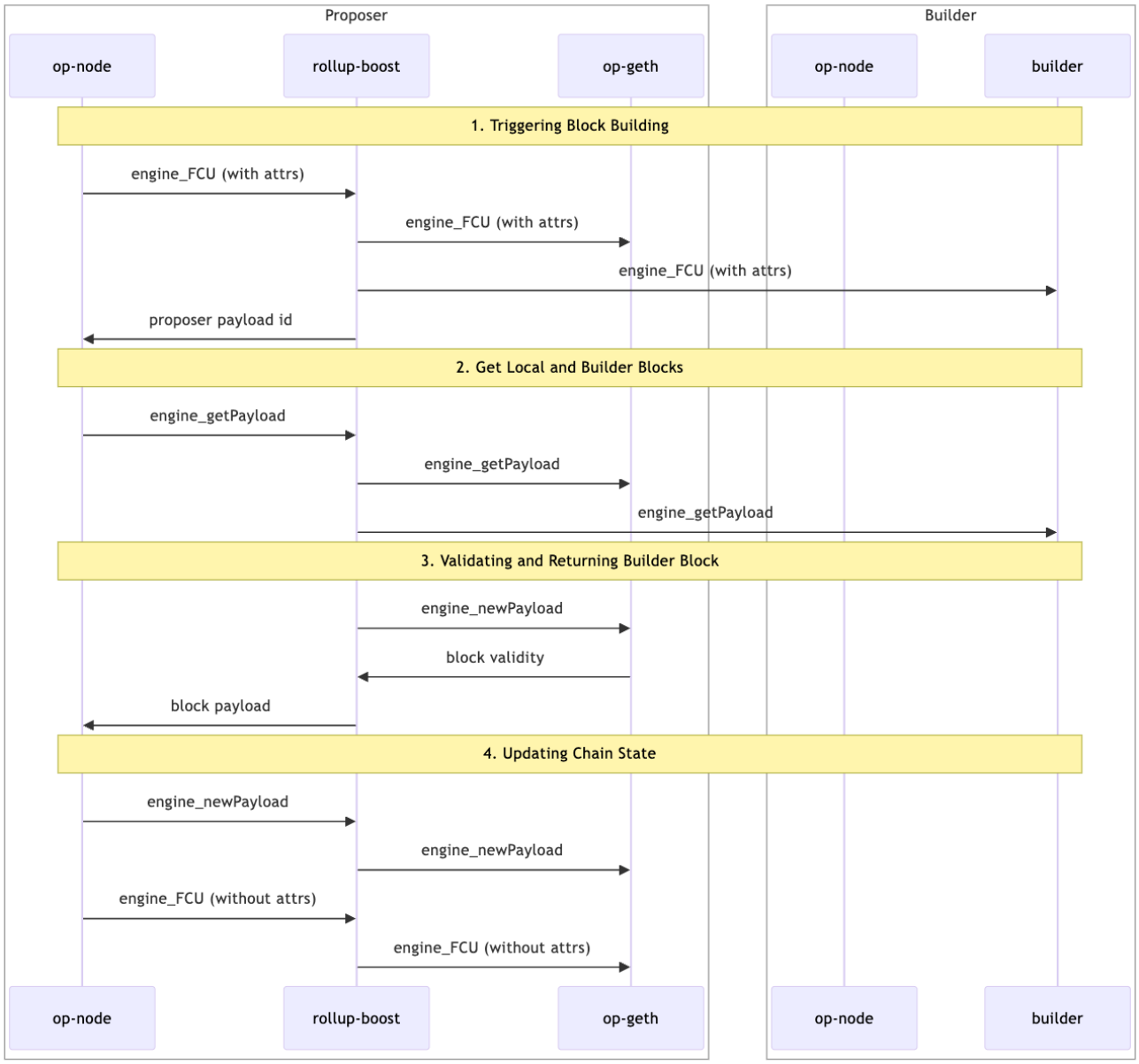

Fig. 2: Sequence diagram extracted from Core system workflow of Rollup Boost

The sequence diagram in Fig. 2 is derived from the Rollup Boost GitHub repository. Basically, it serves as a sidecar component for Optimism Stack chains, enabling external block production by introducing a separation between the *proposer* and *builder* roles. This design allows for greater flexibility in transaction sequencing and block construction. The core workflow operates as follows:

a. **Initiating Block Building**: When the `op-node` initiates block building, it sends an `engine_forkchoiceUpdated` (FCU) call with payload attributes to Rollup Boost. Rollup Boost forwards this call to both the local `op-geth` and the external builder's execution engine.

b. **Fetching Payloads**: Upon receiving an `engine_getPayload` request from the `op-node`, Rollup Boost concurrently requests payloads from both the local `op-geth` and the external builder.

c. **Validating and Selecting Payloads**: Once the builder's payload is received, Rollup Boost validates it by sending an `engine_newPayload` call to the local `op-geth`. If the payload is valid, it is returned to the `op-node`; otherwise, the fallback payload from the local `op-geth` is used.

d. **Updating Chain State**: The `op-node` proceeds to update the chain state by sending an `engine_newPayload` call and another `engine_forkchoiceUpdated` call without payload attributes to Rollup Boost, which forwards these calls to the local `op-geth`.

5.3 HTTP Proxy Layer and Multiplexing Logic in `proxy.rs`

The `proxy.rs` implements a proxy layer using the Tower middleware framework to intercept and route JSON-RPC HTTP requests based on their method. It is the core component responsible for determining whether a request should be:

- Sent to the rollup's local execution engine (`l2_client`),
- Sent to the external builder (`builder_client`),
- Or sent to both, in a *multiplexed* fashion.

(A). Core Components

`ProxyLayer` - A middleware layer that sets up `HttpClient` instances for both the local execution engine and the external builder. It wraps a service (`inner`) and adds health checking via `HealthLayer`.

`ProxyService` - The service implementation that inspects incoming HTTP JSON-RPC requests, reads the method, and applies routing logic. It clones and spawns requests asynchronously when needed.

(B). Routing Logic

The constant string arrays `MULTIPLEX_METHODS` and `FORWARD_REQUESTS` determine how each JSON-RPC method is handled:

- If a method starts with any prefix in `MULTIPLEX_METHODS`, it may be multiplexed (sent to both builder and L2).
- If it's also in `FORWARD_REQUESTS`, it's sent to both the builder and L2.
- Otherwise, it's only handled by the L2 or passed to the `inner` service.

```rust
if MULTIPLEX_METHODS.iter().any(|&m| method.starts_with(m)) {
    if FORWARD_REQUESTS.contains(&method.as_str()) {
        // Asynchronously forward to builder
        // Synchronously forward to L2
    } else {
        // Handled by the inner server service (proxied to the L2 and/or builder execution engine)
    }
} else {
    // Directly forward to L2 engine via HttpClient
}
```

5.4 Main Service Logic: The Role of `server.rs`

The `server.rs` file in the Rollup Boost repository serves as the main entry point for the sidecar service, orchestrating interactions between `op-node`, the local execution client (`op-geth`), and external block builders. It implements the Engine API methods—`engine_forkchoiceUpdated`, `engine_getPayload`, and `engine_newPayload`—to facilitate modular block production in the OP Stack.

(A). Core Responsibilities

The key responsibilities can be summarized in:

- **API Routing and Handling:** Defines HTTP endpoints corresponding to Engine API methods, parses incoming requests from the `op-node`, and routes them to the appropriate handlers.
- **Payload Management:** Coordinates the retrieval and validation of block payloads from both the local `op-geth` and external builders, ensuring selection of the most valid and optimal block.
- **State Coordination:** Maintains internal state to track block building processes, map proposer and builder payload IDs, and manage fallback strategies in case of builder failure.

(B). Core Functions

- `fork_choice_updated_v3`: Processes incoming `forkchoiceUpdatedV3` requests, starts the block building process, and manages payload ID mappings.
- `get_payload`: Handles payload requests (`getPayloadV3` and `getPayloadV4`) by fetching the built block from the local or external builder (preferred) and returning it to the `op-node`.
- `new_payload`: Forwards incoming block payloads (`newPayloadV3` and `newPayloadV4`) to the local `op-geth` for validation and returns the result to the caller.

(C). Execution Mode

Rollup Boost offers configurable execution modes that determine how it interacts with external block builders. These modes can be adjusted at runtime via the Debug API, allowing operators to tailor the behavior of Rollup Boost without restarting the service.

(1). *Available Execution Modes* - The functions below are called to determine the current execution mode and decide whether to forward the request to the external builder or rely exclusively on local engine.

```rust
pub enum ExecutionMode {
    Enabled,
    DryRun,
    Disabled,
    Fallback,
}

impl ExecutionMode {
    fn is_get_payload_enabled(&self) -> bool {
        matches!(self, ExecutionMode::Enabled)
    }

    fn is_disabled(&self) -> bool {
        matches!(self, ExecutionMode::Disabled)
    }

    fn is_fallback_enabled(&self) -> bool {
        matches!(self, ExecutionMode::Fallback)
    }
}
```

- **Enabled**: Rollup Boost forwards all Engine API calls to the external builder. This mode fully integrates the builder into the block production process.
- **DryRun**: All Engine API calls are sent to the builder, except for `engine_getPayload`.
- **Disabled**: No Engine API calls are sent to the builder.
- **Fallback**: Do not use blocks built by the builder. Only blocks from the local `op-geth` are used.

(2). *Fallback Mechanism* - When operating in `Enabled` mode, Rollup Boost attempts to retrieve and validate a payload from the external builder. If this payload is deemed invalid, typically by failing the `engine_newPayload` validation against the local `op-geth`, Rollup Boost falls back to a locally built payload. This fallback ensures that:

- The network remains live and responsive even if the external builder is unavailable or returns bad data.
- The `op-node` always receives a valid payload, regardless of builder reliability.

5.5 RPC Module (`rpc.rs`)

The `rpc.rs` module in Rollup Boost serves as the core interface for forwarding and handling Engine API calls, ensuring seamless integration and coordination among these components.

- The `RpcClient` struct defines the client-side interface for interacting with an execution layer node's Engine API in Rollup Boost. It encapsulates the logic required to make authenticated JSON-RPC requests, relying on a JWT-secured `auth_client` to communicate with the node.
- The `auth_rpc` field specifies the URI of the target RPC server, ensuring that the client knows where to route its requests.
- The `payload_source` field identifies the origin of the payloads, whether from the local execution engine or an external builder, enabling Rollup Boost to distinguish between different sources during block construction and validation.

```rust
pub struct RpcClient {
    auth_client: HttpClient<AuthClientService<HttpBackend>>,
    auth_rpc: Uri,
    payload_source: PayloadSource,
}
```

The module forwards specific Engine API calls, such as `engine_forkchoiceUpdatedV3`, `engine_getPayloadV3`, and `engine_newPayloadV3`, to both the local execution client and external builders. This multiplexing allows for parallel block building and validation processes.

(A). Functionality Overview

The `rpc.rs` module defines the structure and behavior of the RPC client used by Rollup Boost. It establishes connections to the local execution client and external builders using provided URLs and authentication tokens.

6 Risk Rating Methodology

The risk rating methodology used by Nethermind Security follows the principles established by the OWASP Foundation. The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;

b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;

c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;

b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;

c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	**High**	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	**Medium**	**High**
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, Nethermind Security also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;

b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;

c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

7 Issues

7.1 [Medium] JWT unencrypted in HTTP connections

File(s): `src/client/http.rs`

Description: Connections to the L2 and builder nodes are currently allowed to be through HTTP (`http://` URL) or HTTPS (`https://` URL). Assuming the user sets up a HTTP address for a client, and, given that communication over HTTP is unencrypted, the JSON Web Tokens employed for authentication will be also unencrypted. Though with a validity period of only 60 seconds, in non-local setups (e.g., remote builder), an eavesdropper can easily sniff rollup-boost -> builder packets and gather the JWTs in use. With a hold of these, the eavesdropper can then impersonate the rollup-boost.

Recommendation(s): Enforce HTTPS-only connections through the setter `ConnectionBuilder::https_only()`, or warn the user on the use of HTTP in (possible) non-local connections.

Status: Acknowledged

Update from the client: This mirrors the behavior of all major execution clients, which also allow JSON-RPC over HTTP even when JWT authentication is enabled. Rollup Boost follows the same model as existing Ethereum execution clients. HTTP should only used in trusted or local environments.

7.2 [Medium] Server listening on all interfaces by default, without authentication

File(s): `src/cli.rs`

Description: While configurable by the user, the default `0.0.0.0` makes the server accept incoming connections from any interface, possibly allowing an actor to inject malicious packets which are then signed (using rollup-boost's L2/builder JWT secrets) and forwarded to the L2 and builder nodes. The exposure of this issue is aggravated given that there is no use of an authentication mechanism for incoming messages.

Recommendation(s): Consider changing the default listening IP to `localhost` / `127.0.0.1`.

Status: Fixed

Update from the client: Fixed in d3ab9ac

7.3 [Low] Debug API authentication

File(s): `src/debug_api.rs`

Description: The debug API allows runtime modification of critical server state (`ExecutionMode`) and it doesn't seem to have any application-level authentication, security relies only on port accessibility or firewall.

Recommendation(s): Implement mandatory authentication.

Status: Acknowledged

Update from the client: Rollup Boost is not intended to be publicly exposed. Therefore the authentication of the debug API is unnecessary. It's worth mentioning that the same solution is used in op-node's admin RPC.

7.4 [Low] JWT tokens timestamped 60 seconds into the future

File(s): `src/client/auth.rs`

Description: JWT claims used in the engine API are mandated to be valid for +- 60 seconds. However, the current `secret_to_bearer_header()` function creates JWTs issued 60 seconds into the future. This provides a possible eavesdropper to use the JWT tokens for double their intended duration. Moreover, if the system times of the rollup-boost and the receiving node (L2 or builder) are skewed apart more than the latency between these, the receiving node will block the rollup-boost requests due to a perceived `iat` being more than 60 seconds into the future.

Recommendation(s): Remove the addition of the 60 seconds duration from the `iat` calculation. Use only current time.

Status: Fixed

Update from the client: Fixed in 4e0e83b

7.5 [Low] Possible execution_mode inconsistency in `get_payload`

File(s): server.rs

Description: When `get_payload` is called the flow is:

1. Try to get payload from both builder and L2 execution client;

1.1. To decide whether payload should be fetched from external builder, the `execution_mode` is fetched (link) 1.2. The way it is fetched is that the mutex is locked and value is copied 2. Once payload is received, *process it.* 2.1. During payload *processing* the `execution_mode` is fetched yet again by locking the mutex and reading the value.

The issue is that between points 1 & 2, *some time* can pass, and the value of `execution_mode` might be changed in-between. What can happen is:

1. `execution_mode` is set to `Enabled`, which means the builder will start building the block;
2. Actor who has access to `debug_api` can change `execution_mode` to `Disabled`;
3. Builder will return `Ok(Some(payload))`;
4. Even though execution is set to `Disabled` we will send the payload from the builder instead from the L2 Execution Client. This is because:;

```
        let (payload, context) = match (builder_payload, l2_payload) {
            (Ok(Some(builder)), Ok(l2_payload)) => {
                if self.execution_mode().is_fallback_enabled() {
                    // Default to op-geth's payload
                    Ok((l2_payload, PayloadSource::L2))
                } else {
                    Ok((builder, PayloadSource::Builder))
                }
            }
            (_, Ok(l2)) => Ok((l2, PayloadSource::L2)),
            (_, Err(e)) => Err(e),
        }?;
```

We will end up in a branch: `(Ok(Some(builder)), Ok(l2_payload))` but `is_fallback_enabled` will be false.

Recommendation(s): Read `execution_mode` once, at the beginning of `get_payload`.

Status: Fixed

Update from the client: Fixed in PR 198

7.6 [Low] Proxy does not implement rate-limiting

File(s): `src/proxy.rs`

Description: Rate-limiting helps protect systems from abuse, brute-force attacks, and resource overuse by restricting how often a user can make requests. While the rollup-boost main server implements some rate-limiting through `jsonrpsee::Server`, the proxy level (at a level above), does not implement so.

Recommendation(s): Implement a rate-limiting `Service`, add it to the `ProxyService`. See for example this.

Status: Acknowledged

Update from the client: Rollup Boost is not intended to be publicly exposed. Relays or other trusted components should write directly to the proxy service. Chain operators should ensure that rate limiting is enforced elsewhere in the stack.

7.7 [Low] Use of `sync::Cache` in an async context

File(s): `src/server.rs`

Description: `moka` provides both `sync::Cache` and `future::Cache`, for use in multi-threaded and asynchronous contexts, respectively. While slightly more (9%) performant, `sync::Cache` carries some risk of blocking the async executor threads and therefore, the whole rollup-boost operation. See a more detailed discussion here. As of currently, the employed `Cache` methods carry a very low risk of temporarily blocking the executor.

Recommendation(s): Replace `moka::sync::Cache` by `moka::future::Cache`. Performance analysis comparing both.

Status: Fixed

Update from the client: Fixed in 9955e90

7.8 [Info] CI GitHub Action actions-rs unmaintained

File(s): `CI files`

Description: `actions-rs/toolchain` is a GitHub Action that provides `cargo` features (build, test, lint, etc.). As of currently, it is unmaintained and archived, and as such, any possible vulnerabilities will not be fixed.

Recommendation(s): Use `cargo` as already provided by the CI image `ubuntu-latest`, or explore other maintained Rust/ `cargo` GitHub Actions.

Status: Fixed

Update from the client: Fixed in a0c7625

7.9 [Info] Code inside the `is_fallback_enabled()` branch may not be reachable

File(s): `src/servers.rs`

Description: The function `RollupBoostServer.get_payload` ensures the execution mode is enabled before attempting to call the builder. If the execution mode disables payload retrieval (e.g., dry-run mode), the builder is not called.

```rust
async fn get_payload(
    &self,
    payload_id: PayloadId,
    version: Version,
) -> RpcResult<OpExecutionPayloadEnvelope> {
    let l2_client_future = self.l2_client.get_payload(payload_id, version);
    let builder_client_future = Box::pin(async move {
    let execution_mode = self.execution_mode();
    // @audit @audit When fallback mode is enabled, the builder is skipped entirely
    if !execution_mode.is_get_payload_enabled() {
        info!(message = "dry run mode is enabled, skipping get payload builder call");

        // We are in dry run mode, so we do not want to call the builder.
        return Err(ErrorObject::owned(
            INVALID_REQUEST_CODE,
            "Dry run mode is enabled",
            None::<String>,
        ));
    }
    // ...
```

After, the function concurrently awaits both the L2 client and the (potential) builder results using `tokio::join!`. It then matches the outcomes:

- If both the builder and the L2 client succeed (`Ok` results), the function checks if fallback mode is enabled:;

- If fallback mode (`is_fallback_enabled()`) is enabled, the L2 client's payload is chosen. - Otherwise, the builder's payload is used.

- If only the L2 client succeeds while the builder fails or returns no payload, the L2 client's payload is selected instead;

```rust
    let (l2_payload, builder_payload) = tokio::join!(l2_client_future, builder_client_future);
    let (payload, context) = match (builder_payload, l2_payload) {
        (Ok(Some(builder)), Ok(l2_payload)) => {
            if self.execution_mode().is_fallback_enabled() {
                // @audit: This branch will never be executed
                // Default to op-geth's payload
                Ok((l2_payload, PayloadSource::L2))
            } else {
                Ok((builder, PayloadSource::Builder))
            }
        }
        (_, Ok(l2)) => Ok((l2, PayloadSource::L2)),
        (_, Err(e)) => Err(e),
    }?;
```

Let's say the `execution_mode` is changed to fallback (i.e., `self.execution_mode().is_fallback_enabled()` is `true`) before the `get_payload` function is invoked. Then, `builder_client_future` would contains an error meaning that the dry run mode is enabled.

The code inside the `is_fallback_enabled()` branch is unreachable because if fallback mode is active, the builder is never called in the first place — it is skipped earlier when checking `is_get_payload_enabled()`. Although this branch is not executed, it does not cause incorrect behavior. The fallback logic properly defaults to selecting the L2 payload when needed.

Recommendation(s): Review the intended flow to ensure the unused code executes when fallback execution mode is enabled or remove it to reduce its complexity.

Status: Fixed

Update from the client: Fixed in the PR 198

7.10 [Info] Incoming requests have a large size limit

File(s): `src/proxy.rs`

Description: The body of all incoming HTTP requests are first passed through the function `http_helpers::read_body()` which returns the encoded JSON bytes, if valid in the size range. Internally, this function copies the bytes into a new, returned, buffer. Given that the size limit is currently set at `u32::MAX`, much larger than expected requests, extra-large (and likely invalid) requests can be used to cause denial-of-service due to the resources needed to copy all of the incoming data. Moreover, the header size of incoming HTTP requests is not set.

Recommendation(s): Limit the size limit to values closer to expected requests. On the header size, set the maximum header size in order to prevent issues related with oversized headers.

Status: Fixed

Update from the Nethermind Security: The fixing for this issue was identified in PR 221.

7.11 [Info] `boost_sync` flag can cause discrepancies between builder and L2 op-geth

File(s): `src/server.rs`

Description: Here is the relevant part for handling FCU requests:

```
    // TODO: Use _is_block_building_call to log the correct message during the async call to builder
        let (should_send_to_builder, _is_block_building_call) =
            if let Some(attr) = payload_attributes.as_ref() {
                // payload attributes are present. It is a FCU call to start block building
                // Do not send to builder if no_tx_pool is set, meaning that the CL node wants
                // a deterministic block without txs. We let the fallback EL node compute those.
                let use_tx_pool = !attr.no_tx_pool.unwrap_or_default();
                (use_tx_pool, true)
            } else {
                // no payload attributes. It is a FCU call to lock the head block
                // previously synced with the new_payload_v3 call. Only send to builder if boost_sync is enabled
                (self.boost_sync, false)
            };

        let execution_mode = self.execution_mode();
        let trace_id = span.id();
        if let Some(payload_id) = l2_response.payload_id {
            self.payload_trace_context.store(
                payload_id,
                fork_choice_state.head_block_hash,
                payload_attributes.is_some(),
                trace_id,
            );
        }

        if execution_mode.is_disabled() {
            debug!(message = "execution mode is disabled, skipping FCU call to builder", "head_block_hash" =
            ↪  %fork_choice_state.head_block_hash);
        } else if should_send_to_builder {
            let builder_client = self.builder_client.clone();
            tokio::spawn(async move {
                let _ = builder_client
                    .fork_choice_updated_v3(fork_choice_state, payload_attributes.clone())
                    .await;
            });
        } else {
            info!(message = "no payload attributes provided or no_tx_pool is set", "head_block_hash" =
            ↪  %fork_choice_state.head_block_hash);
        }
```

Let's focus on the following scenario:

1. `execution_mode` is `ExecutionMode::Enabled`;
2. FCU is NOT a building block, but an update head;
3. If `self.boost_sync` is disabled, the builder won't receive the update head from the Sequencer; it will have to wait for it from its own L2 node, which can be slow(er);
4. The request for block building comes before the builder has had time to update the head, which results in the builder not building a block;

A similar argument can be made for the `NewPayload` call because it depends on `self.boost_sync`. This may be an even bigger issue because the builder could take too long to receive this new block via P2P (even though it's physically close to the Sequencer).

Recommendation(s): Maybe remove `self.boost_sync` altogether. There is no massive benefit in having this flag; conversely, it complicates logic and may lead to bugs.

If this is not acceptable, we recommend that if `ExecutionMode::Enabled` *overpowers* `self.boost_sync`, i.e., even if `boost_sync` *is* disabled, we forward requests for `ExecutionMode::Enabled`. This later is not the best solution because logic is still unnecessarily complex, and issues can arise (albeit highly unlikely) when `ExecutionMode` is changed and `boost_sync` is disabled.

Status: Fixed

Update from the client: Fixed in the PR 206

7.12 [Info] getPayload requests to builder when FCU `no_tx_pool` is true

File(s): `src/server.rs`

Description: FCUs with attributes have a `no_tx_pool` field. While handling FCUs, the ones with true `no_tx_pool` are not sent to the builder. The code is presented below:

```
    let (should_send_to_builder, _is_block_building_call) =
        if let Some(attr) = payload_attributes.as_ref() {
            // payload attributes are present. It is a FCU call to start block building
            // Do not send to builder if no_tx_pool is set, meaning that the CL node wants
            // a deterministic block without txs. We let the fallback EL node compute those.
            let use_tx_pool = !attr.no_tx_pool.unwrap_or_default();
            (use_tx_pool, true)
        } // ...
    }
    // ...
    else if should_send_to_builder {
        let builder_client = self.builder_client.clone();
        tokio::spawn(async move {
            let _ = builder_client
                .fork_choice_updated_v3(fork_choice_state, payload_attributes.clone())
                .await;
        });
    }
```

However, after this, in `get_payload` the rollup-boost will attempt to fetch a block from the builder even if `no_tx_pool` was true, resulting in an unknown payload error from the builder. The code in the `get_payload` function is presented below:

```
    if !self.payload_trace_context.has_attributes(&payload_id) {
        // block builder won't build a block without attributes
        info!(message = "no attributes found, skipping get_payload call to builder");
        return Ok(None);
    }

    let builder = self.builder_client.clone();
    let payload = builder.get_payload(payload_id, version).await?;
```

Recommendation(s): Do not try to fetch payloads from the builder associated with FCUs with attributes and with true `no_tx_pool`. Leverage/update the use of the trace context accordingly.

Status: Fixed

Update from the client: Fixed in PR 195

7.13 [Best Practice] Concurrency in server

File(s): src/server.rs

Description: Incoming RPC requests are handled by spawning new asynchronous tasks with `tokio::spawn`; the code doesn't show any mechanism to limit the number of concurrently executed tasks.

Recommendation(s): Implement limiting, e.g., Semaphore might be a solution.

Status: Acknowledged

Update from the client: There are instances where op-node will send a quick succession of FCUs following derivation which requires that no rate limiting is in place. Further, Tokio is setup by default with number of background workers equal to number of CPU cores and tokio tasks are efficient to spawn. Relays or other trusted components should write directly to the proxy service. Chain operators should ensure that rate limiting is enforced elsewhere in the stack. We believe this finding should not be in the report.

7.14 [Best Practice] Different conditions for cache insertion/eviction

File(s): `src/server.rs`

Description: While payload IDs are stored in the cache on every FCU call, they are only removed in new-payload calls when `boost_sync` is enabled and the current execution mode is not `Disabled`. If the second set of conditions is not fulfilled, then the cache is only evicted upon its size limit being reached (100 entries).

Recommendation(s): To avoid an unnecessary large cache, only keep track of payload IDs for FCUs which are sent to the builder.

Status: Fixed

Update from the client: Fixed in 07b7f70

7.15 [Best Practice] Do not clone payload in server|fcu_v3

File(s): server.rs

Description: The cloned code in the `server::fork_choice_updated_v3` function (link/src/server.rsL302)) is not needed:

```
let _ = builder_client
    .fork_choice_updated_v3(fork_choice_state, payload_attributes.clone())
    .await;
```

Recommendation(s): We suggest not cloning the `payload_attributes`:

```
- let _ = builder_client
-     .fork_choice_updated_v3(fork_choice_state, payload_attributes.clone())
-     .await;
+ let _ = builder_client
+     .fork_choice_updated_v3(fork_choice_state, payload_attributes)
+     .await;
```

Status: Fixed

Update from the client: Fixed in PR 210

7.16 [Best Practice] Silent failures of `tokio::spawn`

File(s): server.rs

Description: In `fork_choice_updated_v3` and `new_payload` functions, requests to the builder are dispatched asynchronously using `tokio::spawn` to ensure quick server responses. However, there are some concerns:

In the `new_payload` function, the result of the spawned task is explicitly ignored:

```
    tokio::spawn(async move {
        let _ = builder.new_payload(new_payload_clone).await;
    });
```

Recommendation(s): Failures in the spawned tasks should be logged using tracing or captured through metrics to aid in debugging and monitoring.

Status: Acknowledged

Update from the client: The underlying methods (`new_payload_v3` and `new_payload_v4`) are instrumented with `tracing` and they are dispatched early.

7.17 [Best Practice] Using expect and unwrap in the code

File(s): src/client/auth.rs src/client/http.rs

Description: There are unchecked unwrap and expect calls in few non-test paths which can trigger the process to panic! if it fails to unwrap.

Recommendation(s): Replace unwrap/expect with proper error handling

Status: Fixed

Update from the client: Fixed in PR 243. This PR resolved most unwraps and expects in production code. Some unwraps remain in test code. And some expects remain in prod code - but are limited to service initialization or cases where the panic case is guarded against.

7.18 [Best Practice] CI use cargo deny

File(s): .

Description: `cargo deny` is a dependency linter, which analyzes dependencies security advisories, bans, licenses, and sources. If added to CI, such analysis will occur when the respective pipeline is triggered, enabling an alert, if there are any issues with a newly added dependency, or if any new issues popped-up with existing dependencies. There is no apparent dependency scanning or vulnerability management visible in the repository.

Recommendation(s): Add `cargo deny` or any other dependency vulnerability scanner to a CI pipeline.

Status: Fixed

Update from the client: Fixed in PR 249

7.19 [Best Practice] Do not clone payload in rpc|new_payload_v3

File(s): rpc.rs

Description: In rpc.rs|new_payload_v3, the payload is unnecessarily cloned, just to log hash. The average block size on, e.g., Base (which is another op-stack derived chain), is 65KB, while max was 500KB (source), this could easily be avoided.

Recommendation(s): We recommend to apply the following change:

```
- let execution_payload = ExecutionPayload::from(payload.clone());
- let block_hash = execution_payload.block_hash();
+ // Note: this can be hidden behind a function call to improve the readeability
+ let block_hash = payload.payload_inner.payload_inner.block_hash;
```

Note: Similarly for cloning the new_payload here.

Status: Fixed

Update from the client: Fixed in PR 210

7.20 [Best Practice] Logic of forwarding requests should be improved

File(s): proxy.rs

Description: All requests which are to be forwarded should not be treated equally. Specifically, there are eth_ and miner_ requests. If for some reason miner_ succeeds on the L2-EL but fails on the builder (or vice versa), it can result in discrepancy with critical information for consensus, such as GasPrice and GasLimit.

This can lead to a block created by the builder with a higher gas limit than what L2-EL expects. As a result, the L2-EL will mark this block invalid and potentially flag all the future builder blocks as invalid invalid as well, effectively disabling builder functionality.

Recommendation(s): Increase the robustness of the forwarding mechanism. The miner_ should be forwarder to the builder only if the call to EL-L2 succeeds. If the builder fails to process the request, the failure should be logged. In addition, the system should set the execution_status to Disabled to prevent further issues.

Status: Acknowledged

Update from the client: During the audit, issue 7.20 was raised regarding how we forwarded requests in proxy.rs. At the time, all forwarded methods were handled the same way. Nethermind flagged that this could cause an issue if miner_ methods like miner_setGasLimit succeeded on the L2 execution client but failed on the builder, it could lead to inconsistent blocks constraints.

We addressed this in PR 221 by updating the forwarding logic to include retries for miner_setMaxDASize. If the builder request fails, the updated logic set execution_mode to Disabled and continually retried the request. However, this PR introduced a memory leak. The retry logic for miner_setMaxDASize would clone the request and sleep 200ms between attempts. In cases where the call was timing out, this led to a large buildup of in-flight retries and unbounded memory growth.

We fixed this in PR 302 which removes the retry logic for miner_setMaxDASize entirely. Instead, these calls are now treated like any other forwarded request. This is sufficient because op-batcher sends a miner_setMaxDASize update every 10 seconds regardless of the current throttle state, so the builder receives updates regularly even if a single call fails, removing the need for retry logic in the proxy.

Since this fix landed, Flashblocks logic has been added to the codebase and the project has been restructured into a Cargo workspace. The logic previously in scope for the audit is now under crates/rollup-boost.

8 Security Implications of Using Permissioneless Builders

The following issues are only applicable to permissionless builders. They arise because rollup-boost does not check the blocks built by builders but relies solely on **op-geth** to validate these blocks. Furthermore, the system in which Rollup-Boost operates is controlled by Raft consensus, which is not Byzantine resistant.

8.1 MEV extraction attack like the one performed against Mev-Boost

File(s): server.rs

Note: A related attack is described in the analysis of the MEV-Boost Relay Attack.

Description: Under the current setup, an attacker does not even need to exploit the system in a sophisticated way to achieve similar outcomes. They can freely insert or reorder transactions as they wish, without resistance or validation barriers.

All transactions are immediately proxied to the block builders. Builders can observe the transactions and (re)order them in any way they want or insert their MEV-extracting transactions. This is not against the protocol and will pass the *op-geth* validation.

Recommendation(s): We recommend one of the suggestions below to solve it:

- Allow only permissioned builders;
- Encrypted mempools (TEE);
- Maybe not full-blown TEE, but we could only reveal transactions to the builder after they commit to inclusion and ordering of transactions. If they don't respect the commitment, then rollup-boost fallbacks to *op-geth* block;

8.2 Do not blindly default to builder block

File(s): server.rs

Description: The current design blindly defaults to builder-block. This can be problematic as observed in the HA-doc:

> Note that rollup-boost does not currently feature a block selection policy and will optimistically select the builder's block for validation. In the event of a bug in the builder, it is possible valid but undesirable blocks (e.g. empty blocks) are produced. Without a block selection policy, rollup-boost will prefer these blocks over the default execution client. Proper monitoring alerting can help mitigate this but further designs should be explored to introduce safeguards into rollup-boost directly rather than relying on the builder implementation being correct.

Possible scenarios:

- Proxy fails (for whatever reason) to forward some transaction;
- Or, in the future, if builder breaks bad and censors transactions;

In both cases the block produced by the builder will be valid, but _bad_.

Recommendation(s): Since we have both builder-block and the L2-EL block, we could at least pick one which has higher `gas_used` or tx count as the quick solution to some of the problems mentioned above.

8.3 Slow down (or even disable) *op-geth* with extremely large block from builder

File(s): `server.rs`

Description: A bad block builder can create a huge block, which `op-geth` will try to validate. If the block is too large, this can significantly slow down the `op-geth` execution or even cause it to crash altogether.

Here is the code where `op-geth` validates the header, as can be seen on this line, for op-stack chains, validation of the block's `GasLimit` is skipped.

What an attacker (block builder gone bad) can do is prepare this huge block beforehand. It does not even have to be valid, because the Merkle root is compared only after all execution is done (link) to stall the network or knock out the L2-op-geth altogether.

Recommendation(s): Do not trust the block builder unquestioningly. Since we have a block from `op-geth` and a builder, we can perform some quick checks, e.g., that the `GasLimit` produced by the builder is not 100% more, or something similar.

9 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. There are various types of software documentation. Some of the most common types include:

- User manual: A user manual is a document that provides information about how to use the application. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities;
- Code documentation: Code documentation is a document that provides details about the code. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface). It includes details about the methods, parameters, and responses that can be used to interact with the system;
- Testing documentation: Testing documentation is a document that provides information about how the code is tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the application. This type of documentation is critical in ensuring that the system is secure and free from vulnerabilities.

These types of documentation are essential for software development and maintenance. They help ensure that software is properly designed, implemented and tested, and provide a reference for developers who need to modify or maintain the code in the future.

Remarks about the Rollup Boost documentation

The documentation for the Rollup Boost was provided in the folder docs with a high-level description. Moreover, the Rollup Boost team addressed all questions and concerns raised by the Nethermind Security team, providing valuable insights and a comprehensive understanding of the project's technical aspects.

9.1 The code for `get_payload` is not easy to reason about

Code as documentation is crucial for ensuring readability and reducing the effort for program comprehension, especially in complex systems, where implicit logic can easily lead to misunderstandings or errors. The `get_payload` function in `server.rs` exemplifies this: the control flow is hard to trace, which can lead to bugs in the future. By not making the intent and decision-making process explicit in the code, future contributors may misinterpret the expected behavior. Some examples are listed below:

- Inconsistencies in how `execution_mode` is handled (see issue);
- Code inside the `is_fallback_enabled()` branch may not be reachable in certain scenarios (see issue), potentially resulting in future defects.

Recommendation(s): Refactor the `get_payload` function so that there is a clear difference in flow when a block is expected from the builder vs. when it is not. A proposed refactor is attached below, which improves the code by:

a. Ensuring `execution_state` is read once, preventing inconsistencies as described above;
b. Making the control flow more explicit: a block is fetched from the builder only when `ExecutionState::Enabled`;
c. Small performance gain, as we do not have to create a future and call `join` unnecessarily.

The diff below illustrates a suggestion of refactoring:

```
src/server.rs | 81 ++++++
```

```
         let (l2_payload, builder_payload) = tokio::join!(l2_client_future, builder_client_future);
         let (payload, context) = match (builder_payload, l2_payload) {
-            (Ok(Some(builder)), Ok(l2_payload)) => {
-                if self.execution_mode().is_fallback_enabled() {
-                    Ok((l2_payload, PayloadSource::L2))
-                } else {
-                    Ok((builder, PayloadSource::Builder))
-                }
-            }
+            (Ok(Some(builder)), _) => Ok((builder, PayloadSource::Builder)),
             (_, Ok(l2)) => Ok((l2, PayloadSource::L2)),
             (_, Err(e)) => Err(e),
         }?;

-        tracing::Span::current().record("payload_source", context.to_string());
-        counter!("rpc.blocks_created", "source" => context.to_string()).increment(1);
-
-        let inner_payload = ExecutionPayload::from(payload.clone());
-        let block_hash = inner_payload.block_hash();
-        let block_number = inner_payload.block_number();
-
-        info!(
-            message = "returning block",
-            "hash" = %block_hash,
-            "number" = %block_number,
-            %context,
-            %payload_id,
-        );
-        Ok(payload)
+        Ok((payload, context))
     }
 }
```

10 Test Suite Evaluation

10.1 Code Coverage Analysis

Table 3 presents the code coverage with overall percentages exceeding 79% in three categories: function, line, and region metrics. However, branch coverage stands out as notably lower, achieving only 63.64% overall. This discrepancy suggests that **while most functions and lines of code are executed during testing, many conditional branches are not fully explored**. This can leave important edge cases and alternative code paths untested, potentially hiding bugs that only manifest under specific conditions. **Improving branch coverage** should be prioritized to enhance the robustness and reliability of the test suite, where coverage is especially low.

Filename	Function Coverage	Line Coverage	Region Coverage	Branch Coverage
`bin/main.rs`	100.00% (2/2)	100.00% (7/7)	100.00% (2/2)	– (0/0)
`cli.rs`	17.65% (3/17)	60.91% (67/110)	42.20% (46/109)	50.00% (6/12)
`client/auth.rs`	100.00% (6/6)	100.00% (33/33)	100.00% (6/6)	– (0/0)
`client/http.rs`	100.00% (3/3)	100.00% (27/27)	87.50% (7/8)	– (0/0)
`client/rpc.rs`	92.00% (23/25)	92.73% (153/165)	88.89% (72/81)	50.00% (2/4)
`debug_api.rs`	100.00% (14/14)	100.00% (123/123)	90.16% (55/61)	– (0/0)
`health.rs`	100.00% (5/5)	100.00% (15/15)	100.00% (8/8)	100.00% (2/2)
`metrics.rs`	50.00% (3/6)	40.91% (18/44)	38.89% (14/36)	25.00% (1/4)
`proxy.rs`	94.34% (50/53)	96.32% (707/734)	86.76% (249/287)	62.50% (5/8)
`server.rs`	95.38% (62/65)	91.77% (591/644)	85.59% (285/333)	76.67% (23/30)
`tracing.rs`	71.43% (5/7)	72.50% (87/120)	65.12% (28/43)	50.00% (3/6)
Totals	**86.70% (176/203)**	**90.41% (1828/2022)**	**79.26% (772/974)**	**63.64% (42/66)**

Table 3: Test Coverage Summary

A. Final Remarks: Code coverage should be increased

The code has a good level of line coverage, but some branches are not executed in tests. There are several scenarios that should be tested in the application:

- `client/rpc.rs`: The client does not validate or handle the case where `new_payload_v4` returns an invalid status, leaving the `RpcClientError::InvalidPayload` path untested.
- `client/rpc.rs`: The success path for `new_payload_v4` is not exercised, which risks silently breaking correct handling of valid responses.
- `client/rpc.rs`: The handling of `get_payload_v4` receiving a V3 payload is untested; if this happens, the client should emit a version mismatch error.
- `proxy.rs`: The proxy does not have coverage for cases where the HTTP body is unreadable, e.g., truncated or disconnected payloads, which should trigger JSON-RPC error -32700.
- `proxy.rs`: The proxy does not validate or test scenarios where the HTTP body contains syntactically invalid JSON, which should result in a parse error (-32700).
- `proxy.rs`: Valid JSON-RPC requests with unknown method names are not tested and may return incorrect or missing errors instead of the required -32601 error.

Recommendation(s): Consider implementing tests for the scenarios presented above.

10.2 Tests Output

```
cargo test --verbose --features ""
 ...
running 13 tests
test client::rpc::tests::valid_jwt ... ok
test client::rpc::tests::invalid_jwt ... ok
test debug_api::tests::test_debug_client ... ok
test client::rpc::tests::test_invalid_args ... ok
test proxy::tests::test_proxy_service ... ok
test server::tests::test_server ... ok
test proxy::tests::test_forward_miner_set_extra ... ok
test proxy::tests::test_direct_forward_mock_request ... ok
test proxy::tests::test_forward_eth_send_raw_transaction ... ok
test proxy::tests::test_forward_eth_send_raw_transaction_conditional ... ok
test proxy::tests::test_forward_set_max_da_size ... ok
test proxy::tests::test_forward_miner_set_gas_limit ... ok
test proxy::tests::test_forward_miner_set_gas_price ... ok

test result: ok. 13 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 3.05s
     Running `worlrollup-boost/target/debug/deps/rollup_boost-fbfdc57792a0
     9884`
running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
     Running `worlrollup-boost/target/debug/deps/builder_full_delay-bfc796
     ffe204c09c`
running 1 test
test builder_full_delay ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 18.05s
     Running `worlrollup-boost/target/debug/deps/builder_returns_incorrect_
     block-3405f3e0ee2936c7`

running 1 test
test builder_returns_incorrect_block ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 8.94s
     Running `worlrollup-boost/target/debug/deps/execution_mode-ca3f9149ef
     504a5c`

running 1 test
test execution_mode ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 18.76s
     Running `worlrollup-boost/target/debug/deps/no_tx_pool-4d8b833e9d988b
     38`

running 1 test
test no_tx_pool ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 10.78s
     Running `worlrollup-boost/target/debug/deps/remote_builder_down-a30e4c
     733d173d77`

running 1 test
test remote_builder_down ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 21.87s
     Running `worlrollup-boost/target/debug/deps/simple-7feb4dac992462ca`

running 1 test
test test_integration_simple ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 10.73s
```

11 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our Cryptography Research team is dedicated to continuous internal research while fostering close collaboration with external partners. The team has expertise across a wide range of domains, including cryptography protocols, consensus design, decentralized identity, verifiable credentials, Sybil resistance, oracles, and credentials, distributed validator technology (DVT), and Zero-knowledge proofs. This diverse skill set, combined with strong collaboration between our engineering teams, enables us to deliver cutting-edge solutions to our partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

Learn more about us at nethermind.io.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to Nethermind in order that Nethermind could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. Nethermind has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, Nethermind disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. Nethermind does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and Nethermind will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Security Review Report

NM-0693 - World Nullifier Contracts

January 5, 2026

NETHERMIND SECURITY

Contents

1 Executive Summary 2

2 Audited Files 3

3 Summary of Issues 3

4 System Overview 4

5 Risk Rating Methodology 5

6 Issues 6
- 6.1 [Medium] Check BabyJubJub element is on curve . . . 6
- 6.2 [Low] `numPeers` should be exact . . . 6

7 Documentation Evaluation 7

8 Test Suite Evaluation 8

9 About Nethermind 9

1 Executive Summary

This document presents the results of the security review conducted by Nethermind Security for World's Nullifier contracts.

This audit focused on the end-to-end review of the above described changes. **The audit comprises 774** lines of Solidity code. The audit was performed using (a) manual analysis of the codebase, and (b) automated analysis tools.

Along this document, we report 2 points of attention, where 1 is classified as `Medium`, and 1 is classified as `Low` severity. The issues are summarized in Fig. 1.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 summarizes the issues. Section 4 presents the system overview. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 presents the test suite evaluation and automated tools used. Section 9 concludes the document.

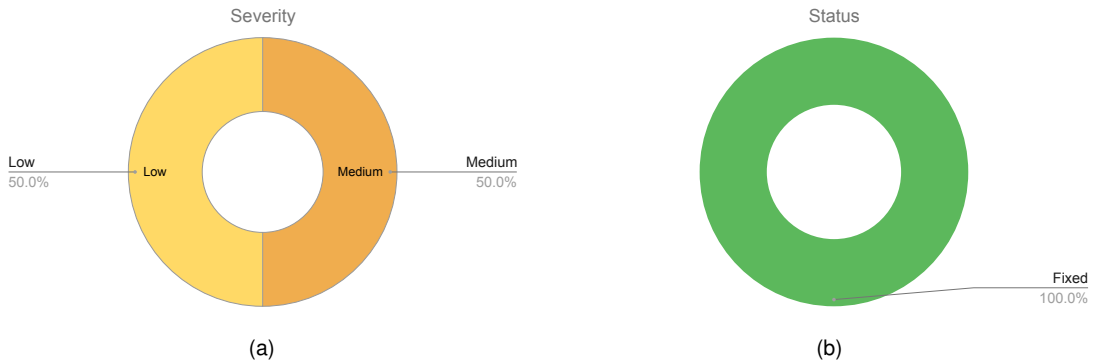

Fig. 1: Distribution of issues: Critical (0), **High** (0), **Medium** (1), **Low** (1), **Undetermined** (0), **Informational** (0), **Best Practices** (0).
Distribution of status: Fixed (2), **Acknowledged** (0), **Mitigated** (0), **Unresolved** (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	October 31, 2025
Final Report	January 5, 2026
Initial Commit Hash	`d3350e4c4ae59d6a99599fb9872b441347c382bd`
Final Commit Hash	`06d1ecb8452c4ba034e746cf2d9cf1d85e262bfc`
Documentation Assessment	High
Test Suite Assessment	High

Summary of Engagement

Dates	Methods	Consultants Engaged	Level of Effort
Sept. 27-Oct. 31-Oct, 2025	Manual	2	2-person weeks

2 Audited Files

	Contract	LoC
1	contracts/src/BabyJubJub.sol	45
2	contracts/src/Groth16VerifierKeyGen13.sol	191
3	contracts/src/Groth16VerifierNullifier.sol	145
4	contracts/src/RpRegistry.sol	340
5	contracts/src/Types.sol	53
Total		**774**

3 Summary of Issues

	Finding	Severity	Update
0	Check BabyJubJub element is on curve	Medium	Fixed
1	`numPeers` should be exact	Low	Fixed

4 System Overview

The RpRegistry contract coordinates a three-round key generation process among registered OPRF peers to securely produce and register a nullifying key. The contract is initialized by an admin, who defines the set of OPRF peers and key generation parameters. Once the system is marked ready, the admin can start a new key generation session for a specific RP ID, creating a state that tracks contributions across rounds.

Following the specification in the paper "A Nullifier Protocol based on a Verifiable, Threshold OPRF", the three round contribution acts as follows:

- **Round 1:** each peer submits a Round1Contribution containing commitment data; the contract aggregates these BabyJubJub points to build a collective key. When all peers contribute, a Round 2 event is emitted.
- **Round 2:** peers submit ciphertexts and accompanying zero-knowledge proofs (verified via the Groth16 verifier). These contributions are validated and stored for all peers. Once every participant completes Round 2, the contract emits a Round 3 event.
- **Round 3:** peers simply acknowledge completion; when all peers have done so, the contract finalizes the process by registering the RP's material — an ECDSA public key and a derived BabyJubJub nullifier key — and cleaning up intermediate data.

Access control in the RpRegistry contract separates administrative and participant privileges to maintain security and order. The owner exclusively registers OPRF peers, while the KeyGenAdmin alone can start new key generation sessions. Only registered OPRF ppers may submit round contributions, verified through strict address-to-peer checks. Modifiers like onlyOwner, onlyAdmin, and isReady enforce correct authorization and initialization, and contract upgrades are restricted to the owner, ensuring controlled governance and secure operation.

The BabyJubJub contract provides arithmetic operations for the BabyJubJub elliptic curve over the BN254 scalar field. It defines core curve parameters and implements point addition and curve validation with modular arithmetic under the field modulus Q. The contract performs safe and deterministic elliptic curve computations, including modular inversion and exponentiation, to support cryptographic protocols that rely on BabyJubJub point operations.

5 Risk Rating Methodology

The risk rating methodology used by Nethermind Security follows the principles established by the OWASP Foundation. The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;

b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;

c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;

b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;

c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	**High**	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	**Medium**	**High**
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, Nethermind Security also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;

b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;

c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [Medium] Check BabyJubJub element is on curve

File(s): `RpRegistry.sol`, `BabyJubJub.sol`

Description: There are multiple instances where an elliptic curve point is used as input, but there is no checking if the point is actually on the curve.

Recommendation(s): Require that inputs to `add(...)`, `submod(...)`, `modInverse(...)` and `expmod(...)` are within the field and inputs in the 'RpRegistry' contract that are treated as elliptic curve elements be checked to be on the curve.

Status: Fixed

Update from the client: Fixed in PR 175.

6.2 [Low] `numPeers` should be exact

File(s): `RpRegistry.sol`

Description: The circuit using the number of peers requires it to be a fixed number, but the initialiser allows for any value for the number of peers that is greater than 2.

Recommendation(s): Restrict peers and threshold to be the corresponding values required by the circuit.

Status: Fixed

Update from the client: Fixed in PR 175.

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- Testing documentation: Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about World Nullifier documentation

The World team provided a clear and comprehensive overview of the World nullifier's contracts during the kick-off call. This was supported with written documentation detailing the different features present in the contracts.

8 Test Suite Evaluation

```
> forge test
Warning: Found unknown config section in foundry.toml: [lint]
This notation for profiles has been deprecated and may result in the profile not being registered in future
 ↪  versions.
Please use [profile.lint] instead or run `forge config --fix`.
[] Compiling...
[] Compiling 56 files with Solc 0.8.27
[] Solc 0.8.27 finished in 13.78s
Compiler run successful!

Ran 5 tests for test/BabyJubjub.t.sol:BabyJubJubTest
[PASS] testAddGeneratorToItself() (gas: 62308)
[PASS] testAddIdentity() (gas: 5985)
[PASS] testGeneratorOnCurve() (gas: 5813)
[PASS] testIdentityPoint() (gas: 5839)
[PASS] testThreeTimes() (gas: 119930)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 4.84ms (3.82ms CPU time)

Ran 1 test for test/Groth16VerifierNullifier.t.sol:NullifierGroth16Verifier
[PASS] testVerifyNullifier() (gas: 276217)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 9.89ms (6.82ms CPU time)

Ran 4 tests for test/Groth16Verifier.t.sol:KeyGen31Groth16Verifier
[PASS] testVerifyAlice() (gas: 350083)
[PASS] testVerifyAnother() (gas: 349951)
[PASS] testVerifyBob() (gas: 350061)
[PASS] testVerifyCarol() (gas: 349885)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 15.14ms (39.64ms CPU time)

Ran 8 tests for test/RpRegistry.t.sol:RpRegistryTest
[PASS] testConstructedCorrectly() (gas: 3269187)
[PASS] testE2E() (gas: 2393916)
[PASS] testInitKeyGenParityWrong() (gas: 21384)
[PASS] testInitKeyGenResubmit() (gas: 348979)
[PASS] testRegisterParticipants() (gas: 3579572)
[PASS] testRegisterParticipantsNotTACEO() (gas: 3286735)
[PASS] testRegisterParticipantsTwice() (gas: 42229)
[PASS] testRegisterParticipantsWrongNumberKeys() (gas: 3295013)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 30.62ms (30.32ms CPU time)

Ran 1 test for test/RpRegistryUpgrade.t.sol:RpRegistryUpgradeTest
[PASS] testUpgrade() (gas: 7786580)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 51.60ms (48.35ms CPU time)

Ran 5 test suites in 54.50ms (112.09ms CPU time): 19 tests passed, 0 failed, 0 skipped (19 total tests)
```

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;

- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to Nethermind in order that Nethermind could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. Nethermind has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, Nethermind disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. Nethermind does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and Nethermind will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Security Review Report

NM-0693 D - World ID Nullifier

January 12, 2026

NETHERMIND SECURITY

Contents

1 Executive Summary **2**

2 Audited Files **3**

3 Summary of Issues **4**

4 System Overview **5**

5 Risk Rating Methodology **7**

6 Issues **8**
- 6.1 [Low] Lack of uniqueness check of party ID's when computing lagrange coefficients . . . 8
- 6.2 [Low] Non-constant time implementation of field arithmetic leading to timing attack vulnerability . . . 8
- 6.3 [Low] EdDSA vulnerable to timing attacks . . . 9
- 6.4 [Low] `their_pk` is not validated during share generation . . . 10
- 6.5 [Info] Debug Assertions in Poseidon2 permutation . . . 10
- 6.6 [Info] G1/G2 point deserialization accepts point at infinity . . . 10
- 6.7 [Info] Some assertions can be made constant . . . 11
- 6.8 [Info] `combine_proofs` asks for `contributing_parties` . . . 11
- 6.9 [Info] Nonce can be reused during share generation . . . 12

7 Documentation Evaluation **13**

8 Test Suite Evaluation **14**
- 8.1 Compilation Output . . . 14
  - 8.1.1 Eddsa BabyJubJub Compilation . . . 14
  - 8.1.2 OPRF Service Compilation . . . 14
  - 8.1.3 Circom-helpers Compilation . . . 14
- 8.2 Tests Output . . . 14
  - 8.2.1 Poseidon2 tests . . . 14
  - 8.2.2 ark-babyjubjub tests . . . 14
  - 8.2.3 OPRF tests . . . 15

9 About Nethermind **17**

1 Executive Summary

This document presents the audit performed by Nethermind Security for the *World ID OPRF Nullifier* code from Nov. 24, 2025 and Dec. 18, 2025. The World Nullifier Service implements a privacy-preserving Verifiable Oblivious Pseudorandom Function for the World ID ecosystem. This audit focuses on validating the correctness of the cryptographic primitives, secret sharing protocols, and distributed proof generation mechanisms that enable relying parties to generate deterministic, unlinkable nullifiers without compromising user privacy or revealing sensitive credential information to any single party.

The audited code comprises 7790 lines of code written in the Rust language. **The audit was performed using** (a) manual analysis of the codebases and (b) automated analysis tools.

Along this document, we report nine point of attention, where four are classified as `Low` and five as `Informational`. **Fig. 1** summarizes the issues identified in the *codebases*.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 summarizes the issues. Section 4 presents the system overview. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 concludes the document. Appendix describes the review of the deployment scripts.

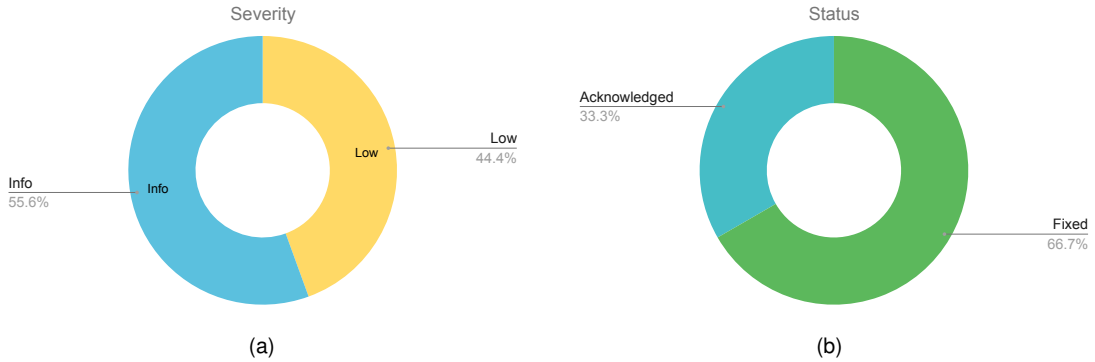

Fig. 1: Distribution of issues: Critical (0), **High** (0), **Medium** (0), **Low** (4), **Undetermined** (0), **Informational** (5), **Best Practices** (0). **Distribution of status: Fixed** (6), **Acknowledged** (3), **Mitigated** (0), **Unresolved** (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	Dec 19, 2025
Final Report	January 12, 2026
Response from Client	Regular responses during audit engagement
OPRF Service Audit Commit	`473667cb16b8d8d6a76b79c4bfdf0b07315db07f`
Poseidon2/BabyJubJub Audit Commit	`ddf370922071c584a43aabe884616abd83520e4f`
Circom Helpers Audit Commit	`334b6b3c0607cd38893b6367b6f28d2ddaa1b570`
Final OPRF Service Audit Commit	`5dd490517f774458ce11894174c45bcffc9dabdb`
Final Poseidon2/BabyJubJub Audit Commit	`1740f6039ce7a43a00748806d4ed232d70207c02`
Final Circom Helpers Audit Commit	`334b6b3c0607cd38893b6367b6f28d2ddaa1b570`
Documentation	OPRF paper
Documentation Assessment	High
Test Assessment	High

Summary of Engagement

Dates	Methods	Consultants Engaged	Level of Effort
Nov. 24-Dec. 18, 2025	Manual	4	16-person weeks

2 Audited Files

	Off-chain code	LoC	Comments	Ratio	Blank	Total
1	oprf-core/src/keygen.rs	340	133	39.12%	64	537
2	oprf-core/src/ddlog_equality/shamir.rs	255	66	25.88%	37	358
3	oprf-core/benches/oprf.rs	244	2	0.82%	25	271
4	oprf-core/src/ddlog_equality.rs	161	48	29.81%	18	227
5	oprf-core/src/oprf.rs	147	30	20.41%	31	208
6	oprf-core/src/dlog_equality.rs	112	27	24.11%	18	157
7	oprf-core/src/shamir.rs	86	44	51.16%	10	140
8	oprf-core/src/lib.rs	6	7	116.67%	0	13
9	oprf-types/src/crypto.rs	166	66	39.76%	33	265
10	oprf-types/src/api/v1.rs	71	35	49.30%	10	116
11	oprf-types/src/chain.rs	56	48	85.71%	11	115
12	oprf-types/src/lib.rs	50	21	42.00%	13	84
13	oprf-types/src/api.rs	1	5	500.00%	0	6
19	ark-babyjubjub/src/lib.rs	15	15	100.00%	2	32
20	ark-babyjubjub/src/constraints/curves.rs	7	1	14.29%	3	11
21	ark-babyjubjub/src/constraints/fields.rs	6	1	16.67%	2	9
22	ark-babyjubjub/src/fields/fr.rs	6	0	0.00%	1	7
23	ark-serde-compat/src/lib.rs	523	301	57.55%	63	887
24	ark-serde-compat/src/babyjubjub.rs	214	64	29.91%	27	305
25	ark-serde-compat/src/impl_macro.rs	238	0	0.00%	23	261
26	circom-types/src/groth16/zkey.rs	592	77	13.01%	57	726
27	circom-types/src/groth16/verification_key.rs	213	13	6.10%	17	243
28	circom-types/src/groth16/zkey_to_ark.rs	200	9	4.50%	16	225
29	circom-types/src/groth16/proof.rs	105	9	8.57%	9	123
30	circom-types/src/groth16/public_input.rs	64	4	6.25%	10	78
31	circom-types/src/groth16/mod.rs	20	1	5.00%	1	22
32	circom-types/src/plonk/zkey.rs	305	87	28.52%	23	415
33	circom-types/src/plonk/verification_key.rs	204	18	8.82%	15	237
34	circom-types/src/plonk/proof.rs	189	21	11.11%	14	224
35	circom-types/src/plonk/mod.rs	12	1	8.33%	2	15
36	circom-types/src/bin/convert-zkey-to-ark.rs	73	8	10.96%	14	95
37	circom-types/src/r1cs.rs	250	50	20.00%	40	340
38	circom-types/src/traits.rs	189	27	14.29%	36	252
39	circom-types/src/witness.rs	117	15	12.82%	17	149
40	circom-types/src/binfile.rs	83	6	7.23%	12	101
41	circom-types/src/lib.rs	160	17	10.62%	21	198
42	groth16/src/lib.rs	195	28	14.36%	20	243
43	groth16/src/reduction.rs	241	16	6.64%	17	274
44	groth16-material/src/circom.rs	284	137	48.24%	38	459
45	groth16-material/src/lib.rs	12	9	75.00%	2	23
1	poseidon2/src/bn254/t16.rs	328	24	7.32%	10	362
2	poseidon2/src/bn254/t12.rs	276	24	8.70%	12	312
3	poseidon2/src/bn254/t8.rs	219	24	10.96%	12	255
4	poseidon2/src/bn254/t4.rs	211	26	12.32%	13	250
5	poseidon2/src/perm.rs	168	36	21.43%	18	222
6	poseidon2/src/bn254/t3.rs	153	24	15.69%	12	189
7	poseidon2/src/bn254/t2.rs	137	22	16.06%	13	172
8	poseidon2/src/bn254.rs	60	18	30.00%	9	87
9	poseidon2/benches/poseidon2.rs	22	0	0.00%	4	26
10	poseidon2/src/lib.rs	4	17	425.00%	1	22
	Total	**7790**	**1682**	**21.59%**	**876**	**10348**

3 Summary of Issues

	Finding	Severity	Update
1	Lack of uniqueness check of party ID's when computing lagrange coefficients	Low	Fixed
2	Non-constant time implementation of field arithmetic leading to timing attack vulnerability	Low	Acknowledged
3	EdDSA vulnerable to timing attacks	Low	Fixed
4	`their_pk` is not validated during share generation	Low	Fixed
5	Debug Assertions in Poseidon2 permutation	Info	Acknowledged
6	G1/G2 point deserialization accepts point at infinity	Info	Acknowledged
7	Some assertions can be made constant	Info	Fixed
8	`combine_proofs` asks for `contributing_parties`	Info	Fixed
9	Nonce can be reused during share generation	Info	Fixed

4 System Overview

This system implements an Oblivious PseudoRandom Function-based nullifier construction designed for threshold security and zero-knowledge compatibility. The deterministically derive unlinkable nullifiers from private inputs, while allowing a distributed set of servers to jointly evaluate the OPRF without revealing the underlying secret key or the client input.

The nullifier construction relies on SNARK-friendly cryptography to ensure efficient verification inside zero-knowledge circuits. The poseidon2 crate implements the Poseidon2 permutation over the BN254 scalar field, Poseidon2 is used to hash OPRF transcripts, derive nullifiers, and construct Poseidon-based Merkle trees used to commit to nullifier sets or protocol state.

BabyJubJub serves as the elliptic-curve group for the OPRF. The system uses arkworks-compatible BabyJubJub implementations for group operations and constraint-system support, and EdDSA-BabyJubJub for signing and authentication of protocol artifacts when required.

The oprf-core crate implements the OPRF-based nullifier protocol. To enforce correctness in a distributed setting, oprf-core implements discrete-log equality proofs, including double DLog equality constructions combined with Shamir secret sharing. These mechanisms ensure that each server's OPRF contribution is consistent with its secret share without revealing the share itself.

As can be seen in the diagram (Fig.2), the user hashes the identity commitment, relaying party ID and the action it wants to perform(e.g. a vote) to obtain a query. this query is blinded over the BabyJubJub elliptic curve with a random scalar β. It then send the blinded query to all servers, to compute an OPRF response ($C_i = B \cdot x_i$), and generates random commitments for the proof. These commitments are then aggregated through Lagrange interpolation reconstructing $B \cdot x$.

Over round 2, each server computes a challenge hash and generate proof shares, these shares are collected by the client and combines it into a single Chaum-Pedersen proof for which it can verify againt the public key. If everything works out, the client unblinds by multiplying β^{-1}, this is hashed one final time to produce the nullifier.

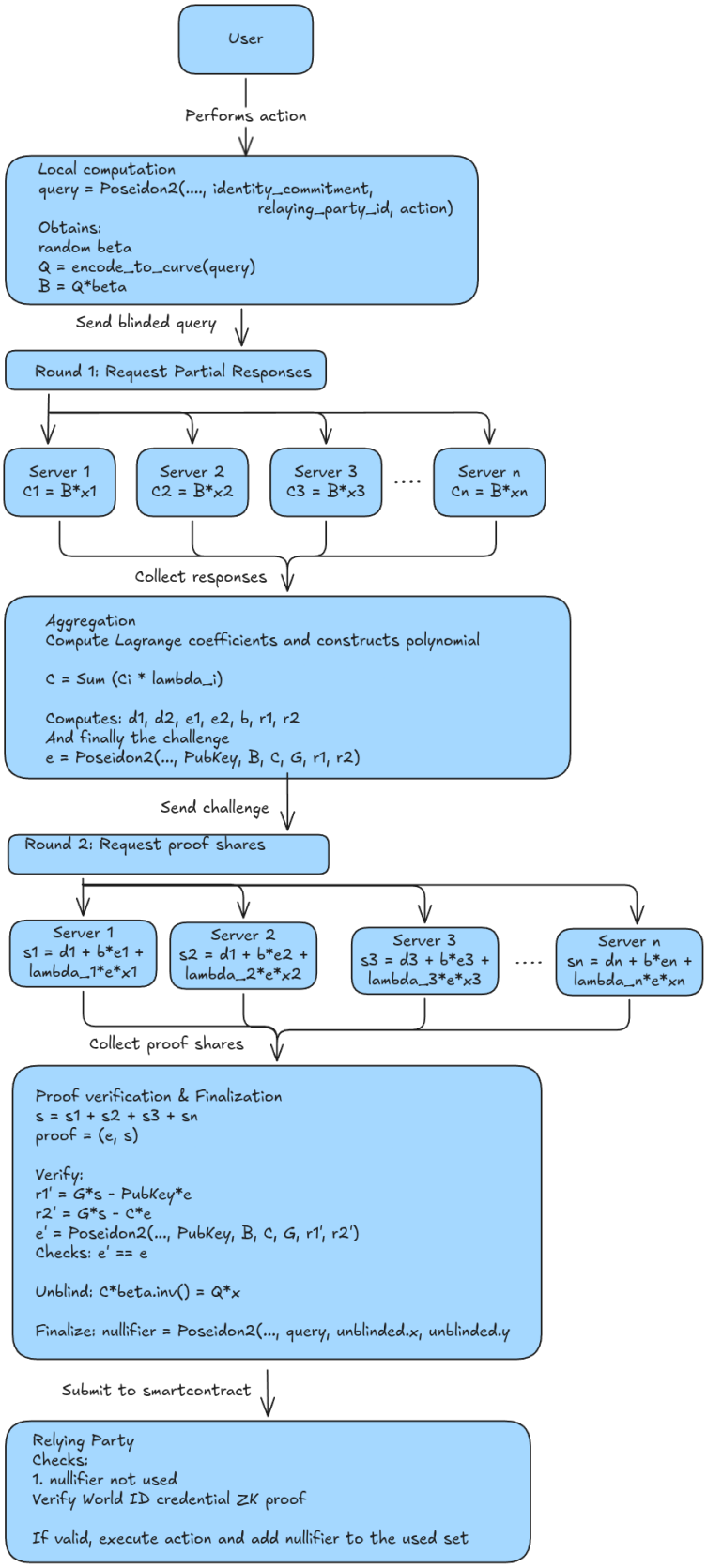

Fig. 2: OPRF core flow

5 Risk Rating Methodology

The risk rating methodology used by Nethermind Security follows the principles established by the OWASP Foundation. The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;

b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;

c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;

b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;

c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Likelihood		
		Low	**Medium**	**High**
Impact	**High**	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined

To address issues that do not fit a High/Medium/Low severity, Nethermind Security also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;

b) **Best Practice** findings are used when some piece of code does not conform with smart contract and circuit development best practices;

c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [Low] Lack of uniqueness check of party ID's when computing lagrange coefficients

File(s): oprf-core/src/ddlog_equality/shamir.rs

Description: In `combine_commitments(...)` it is required to compute the lagrange coefficients for the provided contributing parties. However, there is no check that the contributing parties are unique. This means it is possible to use duplicate party IDs and commitments. Maliciously this can be used to inflate the number of shares and produce an incorrect secret from the secret sharing.

Recommendation(s): Before computing the lagrange coefficients for the contributing parties, check that:

```
set(contributing_parties).len() == contributing_parties.len();
```

This will ensure only unique party IDs are being used in the commitment combination.

Status: Fixed

Update from the client: Fixed in PR #339.

6.2 [Low] Non-constant time implementation of field arithmetic leading to timing attack vulnerability

File(s): oprf-core/src/ddlog_equality/shamir.rs, oprf-core/src/oprf.rs, oprf-core/src/shamir.rs, oprf-core/src/dlog_equality.rs

Description: The underlying field arithmetic used for the following functions uses branching conditions that branch based on the secret values used. Here we provide a list of key functions we believe are vulnerable to such attacks and why:

1. `shamir.rs/dh_key_derivation(...)` performs multiplication of a scalarfield element and an affine element performing group multiplication for affine coordinates using the double-and-add method;
2. `oprf.rs/unblind_response(...)` is used to unblind the server response using the prepared blinding factor performed with the double-and-add method;
3. `dlog_equality.rs/partial_commitments(...)` performs multiplication over the secret share `x_share` using the double-and-add method;
4. `shamir.rs/single_lagrange_from_coeff(...)` and `shamir.rs/evaluate_poly(...)` performs field multiplication for a prime field, the `ScalarField` nor `BaseField` types do not implement constant-time multiplication;

Recommendation(s): Implement constant time field arithmetic.

Status: Acknowledged

Update from the client: Most low-level field arithmetic operations in ark-ff are constant time, with a few exceptions like the subtraction that does a branching on potential overflows. We accept this for now and will re-evaluate in the future if we want to address this further by changing out the underlying field arithmetic completely.

6.3 [Low] EdDSA vulnerable to timing attacks

File(s): eddsa-babyjubjub/src/lib.rs

Description: Timing attacks are side channel attacks that enable attackers to infer values of secret keys by noticing changes in the time it takes to perform cryptographic operations. As stated by RFC 8032, such attacks stem from the secret key influencing the set of instructions and memory accesses executed by the code. It was found that the implementation of EdDSA is vulnerable to such attacks after running `dudect` to measure the signature generation speed against a fixed key and a random key. Specifically, the code run was with 1 million samples:

```rust
/// Differential time analysis of signing, fixed key & msg vs. fixed msg & random key
fn eddsa_sign_ct_vary_sk(runner: &mut CtRunner, rng: &mut BenchRng) {
    let msg = BaseField::from(0u64);

    let mut sk_rng = rand::thread_rng();
    let mut keys = Vec::with_capacity(N_SAMPLES);
    let mut classes = Vec::with_capacity(N_SAMPLES);

    for _ in 0..N_SAMPLES {
        if rng.next_u32() & 1 == 0 {
            // Left: fixed key
            keys.push(EdDSAPrivateKey::from_bytes([0x13; 32]));
            classes.push(Class::Left);
        } else {
            // Right: random key
            keys.push(EdDSAPrivateKey::random(&mut sk_rng));
            classes.push(Class::Right);
        }
    }

    for (class, sk) in classes.into_iter().zip(keys.into_iter()) {
        runner.run_one(class, || {
            let sig = sk.sign(msg);
            core::hint::black_box(sig);
        });
    }
}
```

It was found that this issue stems from the branching condition found in the scalar multiplication in `ark-ec/src/twisted_-edwards/mod.rs` which implements affine multiplication using the double-and-add method:

```rust
/// Default implementation of group multiplication for affine coordinates
fn mul_affine(base: &Affine<Self>, scalar: &[u64]) -> Projective<Self> {
    let mut res = Projective::<Self>::zero();
    for b in ark_ff::BitIteratorBE::without_leading_zeros(scalar) {
        res.double_in_place();
        if b {
            res += base
        }
    }

    res
}
```

The attack is possible here as an attacker can measure how many times a value is added and how many times it is multiplied. This is because the time to compute a multiplication and the time to complete an addition is different, allowing the attacker to infer information about the key.

Recommendation(s): We recommend using a constant time implementation of the fields to prevent this attack. However, as far as we understand, there does not exist a constant time implementation of the BabyJubJub curve enabling the usage of constant time curve operations. Arkworks does not implement constant time field operations. Thus, the suggestion is to reimplement a rust crate like Sylow, which uses constant time field operations for BN254, for BJJ.

For Arkworks, a solution could be to implement constant time scalar multiplication to bypass this through the Montgomery ladder method. Various solutions are provided in the paper Elliptic Curves and Side-Channel Analysis.

Status: Fixed

Update from the client: Fixed in PR #11

6.4 [Low] their_pk is not validated during share generation

File(s): oprf-core/src/keygen.rs

Description: gen_share public function accepts their_pk as an argument, but it does not validate it being on-curve.

There is no code calling this function in the audit scope (outside of local tests). There is a validation performed if their_pk is obtained specifically via checked versions of deserialize_* methods of CanonicalDeserialize trait.

If obtained by other means though, there's no safeguard against their_pk being off-curve, thus making private key vulnerable to invalid curve attack.

Recommendation(s): We recommend checking their_pk being on curve and in the correct subgroup inside the gen_share function itself.

Status: Fixed

Update from the client: Fixed in PR #339.

6.5 [Info] Debug Assertions in Poseidon2 permutation

File(s): eddsa-babyjubjub/poseidon2/src/perm.rs

Description: The Poseidon2 permutation file contains some debug assertions. These are not part of the release build and will enable users to instantiate Poseidon2 internal matrices with different parameters than the recommended.

```
/// The matrix multiplication in the internal rounds of the Poseidon2 permutation.
    pub fn matmul_internal(&self, input: &mut [F; T]) {
        match T {
            2 => {
                // Matrix [[2, 1], [1, 3]]
                debug_assert_eq!(self.mat_internal_diag_m_1[0], F::one());
                debug_assert_eq!(self.mat_internal_diag_m_1[1], F::from(2u64));
                let sum = input[0] + input[1];
                input[0] += &sum;
                input[1].double_in_place();
                input[1] += sum;
            }
            3 => {
                // Matrix [[2, 1, 1], [1, 2, 1], [1, 1, 3]]
                debug_assert_eq!(self.mat_internal_diag_m_1[0], F::one());
                debug_assert_eq!(self.mat_internal_diag_m_1[1], F::one());
                debug_assert_eq!(self.mat_internal_diag_m_1[2], F::from(2u64));
            [...]
```

Status: Acknowledged

Update from the client: They acknowledged they are aware of the issue. But it is a minor one, as the library is intended for internal use. Additionally, they would like to explore verification at compile time to avoid incurring additional runtime costs for the assertions. The issue is presented in an informational manner to help keep track of the problem.

6.6 [Info] G1/G2 point deserialization accepts point at infinity

File(s): ark-serde-compat/src/lib.rs

Description: serde deserialization of G1 points via intended [serde(deserialize_with = "ark_serde_compat::CURVE::deserialize_g1")] boils down to G1Visitor called with const CHECK == true. This CHECK enforces the deserialized point to be on curve and in the subgroup of the correct order, otherwise deserialization will error out. However, it does explicitly allow point at infinity (a.k.a zero point).

Using a point at infinity in certain context may lead to the soundness property of a proof system being broken.

All of the above applies to G2 points as well.

This deserialization routine is used e.g. in VerificationKey of circom_types::plonk and circom_types::groth16, where no additional validation is being done.

Recommendation(s): Care should be taken when using this deserialization, with additional validation performed accordingly on deserialized entities where this might impact security guarantees.

Status: Acknowledged

Update from the client: They say that the validation is done on the contract side, so PaI won't reach the deserialization.

6.7 [Info] Some assertions can be made constant

File(s): eddsa-babyjubjub/poseidon2/src/perm.rs

Description: The Poseidon2 permutation file contains some assertions that can be updated to be done at compile time.

```
/// Creates a new instance of the Poseidon2 permutation with given parameters
    pub const fn new(
        mat_internal_diag_m_1: [F; T],
        round_constants_external: [[F; T]; ROUNDS_F],
        round_constants_internal: [F; ROUNDS_P],
    ) -> Self {
        assert!(T == 2 || T == 3 || ((T <= 24) && (T % 4 == 0)));
        assert!(D % 2 == 1);
        assert!(ROUNDS_F % 2 == 0);

        Self {
            mat_internal_diag_m_1,
            round_constants_external,
            round_constants_internal,
        }
    }
```

These assertions only involved information known at compile time. They can be defined within a const block (stabilized since Rust 1.79)

```
pub const fn new(
    mat_internal_diag_m_1: [F; T],
    round_constants_external: [[F; T]; ROUNDS_F],
    round_constants_internal: [F; ROUNDS_P],
) -> Self {
    const {
            assert!(T == 2 || T == 3 || ((T <= 24) && (T % 4 == 0)));
            assert!(D % 2 == 1);
                        assert!(ROUNDS_F % 2 == 0);
                }
          // [...]
}
```

Alternatively, use static_assertions.

This ensures the checks are done at compile time instead of runtime:

- Avoiding wrong configurations for Poseidon2;
- Transforms a potential panic into a compilation error;

Status: Fixed

Update from the client: Fixed in PR #10

6.8 [Info] `combine_proofs` asks for `contributing_parties`

File(s): oprf-core/src/ddlog_equality/shamir.rs

Description: One argument to `DLogEqualityCommitments::combine_proofs` and `DLogCommitmentsShamir::combine_proofs` methods is `contributing_parties` listing parties that contributed to proof generation. However, `DLogEqualityCommitments` (and by extension, its wrapper `DLogCommitmentsShamir`) already contains `contributing_parties` vector.

Having `contributing_parties` argument is unnecessary and introduces risk of misusing it.

Recommendation(s): We recommend removing `contributing_parties` argument from `combine_proofs` and pass in.

Status: Fixed

Update from the client: Fixed in PR #339.

6.9 [Info] Nonce can be reused during share generation

File(s): `oprf-core/src/keygen.rs`

Description: `gen_share` public function accepts nonce as an argument. While there's no code calling this function in the audit scope (outside of local tests), it opens the opportunity for it to be misused by passing the same nonce multiple times. If allowed, it might make party's secret key vulnerable.

Recommendation(s): We recommend passing in the secure RNG and generating nonce in the `gen_share` function itself.

Status: Fixed

Update from the client: Fixed in PR #339.

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Soource code can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the Source code's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the source code. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- Testing documentation: Testing documentation is a document that provides information about how the source code was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the source code. This type of documentation is critical in ensuring that the source code is secure and free from vulnerabilities.

Remarks about the World nullifier service

The **World** team thoroughly addressed all questions and concerns raised by the **Nethermind Security** team, providing valuable insights and a comprehensive understanding of the project's technical aspects.

8 Test Suite Evaluation

Initially tests were failing due to broken dependencies. This was fixed here

8.1 Compilation Output

8.1.1 Eddsa BabyJubJub Compilation

```
cargo build
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.03s
```

8.1.2 OPRF Service Compilation

```
cargo build
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.19s
warning: the following packages contain code that will be rejected by a future version of Rust: russh v0.54.5
note: to see what the problems were, use the option `--future-incompat-report`, or run `cargo report
 →  future-incompatibilities --id 3`
```

8.1.3 Circom-helpers Compilation

```
cargo build
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.10s
```

8.2 Tests Output

8.2.1 Poseidon2 tests

```
cargo test
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0.03s
     Running unittests src/lib.rs (~/eddsa-babyjubjub/target/debug/deps/taceo_poseidon2-c9098a60311cc71c)

running 13 tests
test bn254::t2::tests::poseidon2_bn254_t2_kat1 ... ok
test bn254::t3::tests::poseidon2_bn254_t3_kat1 ... ok
test bn254::t4::tests::poseidon2_bn254_t4_kat1 ... ok
test bn254::t16::tests::poseidon2_bn254_t16_kat1 ... ok
test bn254::t4::tests::poseidon2_bn254_t4_kat2 ... ok
test bn254::t12::tests::poseidon2_bn254_t12_kat1 ... ok
test bn254::t8::tests::poseidon2_bn254_t8_kat1 ... ok
test bn254::t2::tests::poseidon2_bn254_t2_consistent_perm ... ok
test bn254::t3::tests::poseidon2_bn254_t3_consistent_perm ... ok
test bn254::t4::tests::poseidon2_bn254_t4_consistent_perm ... ok
test bn254::t8::tests::poseidon2_bn254_t8_consistent_perm ... ok
test bn254::t16::tests::poseidon2_bn254_t16_consistent_perm ... ok
test bn254::t12::tests::poseidon2_bn254_t12_consistent_perm ... ok

test result: ok. 13 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.01s

   Doc-tests taceo_poseidon2

running 2 tests
test poseidon2/src/bn254.rs - bn254 (line 12) ... ignored
test poseidon2/src/lib.rs - (line 12) ... ignored

test result: ok. 0 passed; 0 failed; 2 ignored; 0 measured; 0 filtered out; finished in 0.00s
```

8.2.2 ark-babyjubjub tests

```
cargo test
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0.02s
     Running unittests src/lib.rs (~/eddsa-babyjubjub/target/debug/deps/taceo_ark_babyjubjub-8cf5e12d0d642b54)

running 52 tests
test curves::tests::te::test_montgomery_conversion_test ... ok
```

```
test fields::tests::fq::test_montgomery_config ... ok
test fields::tests::fq::test_fft ... ok
test fields::tests::fq::test_constants ... ok
test fields::tests::fq::test_sum_of_products_edge_case ... ok
test curves::tests::te::test_te_properties ... ok
test fields::tests::fq::test_mul_by_base_field_elem ... ok
test fields::tests::fr::test_constants ... ok
test fields::tests::fr::test_fft ... ok
test fields::tests::fq::test_serialization ... ok
test fields::tests::fr::test_montgomery_config ... ok
test fields::tests::fr::test_mul_by_base_field_elem ... ok
test fields::tests::fq::test_frobenius ... ok
test fields::tests::fr::test_frobenius ... ok
test fields::tests::fr::test_serialization ... ok
test fields::tests::fq::test_mul_properties ... ok
test fields::tests::fr::test_mul_properties ... ok
test fields::tests::fr::test_sum_of_products_edge_case ... ok
test fields::tests::fq::test_pow ... ok
test fields::tests::test_fq_add ... ok
test fields::tests::test_fq_add_one ... ok
test fields::tests::test_fq_div ... ok
test fields::tests::test_fq_double_in_place_thrice ... ok
test fields::tests::test_fq_from_str ... ok
test fields::tests::test_fq_generate_random_ed_on_bn254_point ... ok
test fields::tests::test_fq_legendre ... ok
test fields::tests::test_fq_mul ... ok
test fields::tests::test_fq_square_in_place ... ok
test fields::tests::test_fq_sub ... ok
test fields::tests::test_fq_triple_mul ... ok
test fields::tests::test_fr_add ... ok
test fields::tests::test_fr_from_str ... ok
test fields::tests::test_fr_mul ... ok
test fields::tests::fq::test_sqrt ... ok
test curves::tests::te::test_sub_properties ... ok
test curves::tests::te::test_mixed_addition ... ok
test fields::tests::fr::test_pow ... ok
test fields::tests::fr::test_sqrt ... ok
test curves::tests::te::test_chunked_pippenger ... ok
test curves::tests::te::test_add_properties ... ok
test curves::tests::te::test_hashmap_pippenger ... ok
test fields::tests::fq::test_sum_of_products_tests ... ok
test fields::tests::fr::test_sum_of_products_tests ... ok
test curves::tests::te::test_var_base_msm ... ok
test curves::tests::te::test_serialization ... ok
test curves::tests::te::test_cofactor_ops ... ok
test curves::tests::te::test_affine_conversion ... ok
test fields::tests::fq::test_sub_properties ... ok
test fields::tests::fr::test_sub_properties ... ok
test fields::tests::fq::test_add_properties ... ok
test fields::tests::fr::test_add_properties ... ok
test curves::tests::te::test_mul_properties ... ok

test result: ok. 52 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 36.53s

   Doc-tests taceo_ark_babyjubjub

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
```

8.2.3 OPRF tests

```
cargo test
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0.10s
     Running unittests src/lib.rs
      ↪  (~/nullifier-oracle-service/target/debug/deps/taceo_oprf_core-ab9ebd41115a6c77)

running 16 tests
test oprf::mappings::tests::test_ct_is_zero ... ok
test oprf::mappings::tests::test_map_to_curve_twisted_edwards ... ok
test oprf::mappings::tests::test_encode_to_curve ... ok
test oprf::mappings::tests::test_hash_to_curve ... ok
test oprf::mappings::tests::test_inv0 ... ok
test oprf::tests::test_oprf_determinism ... ok
```

```
test oprf::mappings::tests::test_ct_is_square ... ok
test dlog_equality::tests::test_dlog_equality ... ok
test oprf::mappings::tests::test_map_to_curve_twisted_edwards_rand ... ok
test keygen::tests::test_distributed_keygen_3_1 ... ok
test ddlog_equality::shamir::tests::test_distributed_dlog_equality_shamir_3_1 ... ok
test keygen::tests::test_reshare_3_1 ... ok
test oprf::tests::test_oprf_with_proof ... ok
test ddlog_equality::shamir::tests::test_distributed_dlog_equality_shamir_31_15 ... ok
test keygen::tests::test_distributed_keygen_31_15 ... ok
test keygen::tests::test_reshare_31_15 ... ok

test result: ok. 16 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 2.59s

   Doc-tests taceo_oprf_core

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
```

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our Cryptography Research team is dedicated to continuous internal research while fostering close collaboration with external partners. The team has expertise across a wide range of domains, including cryptography protocols, consensus design, decentralized identity, verifiable credentials, Sybil resistance, oracles, and credentials, distributed validator technology (DVT), and Zero-knowledge proofs. This diverse skill set, combined with strong collaboration between our engineering teams, enables us to deliver cutting-edge solutions to our partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

Learn more about us at nethermind.io.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to Nethermind in order that Nethermind could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. Nethermind has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, Nethermind disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. Nethermind does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and Nethermind will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Security Review Report

NM-0411-0817 - World Nullifier Contracts

February 18, 2026

NETHERMIND SECURITY

Contents

1 Executive Summary **2**

2 Audited Files **3**

3 Summary of Issues **3**

4 System Overview **4**
4.1 Access Control . . . 4

5 Risk Rating Methodology **5**

6 Issues **6**
6.1 [Medium] participants can reuse past rounds contributions from other parties . . . 6
6.2 [Low] Deleted `oprfKeyId` can be initiated again . . . 7
6.3 [Low] `numPeers` should be exact . . . 8

7 Documentation Evaluation **9**

8 Test Suite Evaluation **10**
8.1 Tests Output . . . 10
8.2 Automated Tools . . . 11
8.2.1 AuditAgent . . . 11

9 About Nethermind **12**

1 Executive Summary

This document presents the results of the security review conducted by Nethermind Security for TACEO's OPRF contracts. The OPRF Key Registry contract implements the three-round key-generation and resharing protocol described in A Nullifier Protocol based on a Verifiable, Threshold OPRF. It can run multiple instances of this protocol, tracking shares and commitments across rounds and ultimately deriving the corresponding public key once the protocol completes. **The audit comprises 1122** lines of the Solidity code. **The audit was performed using** (a) manual analysis of the codebase, and (b) automated analysis tools.

Along this document, we report 3 points of attention, where one is classified as `Medium`, and two are classified as `Low`. The issues are summarized in Fig. 1.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 presents the summary of issues. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 presents the test suite evaluation and automated tools used. Section 9 concludes the document.

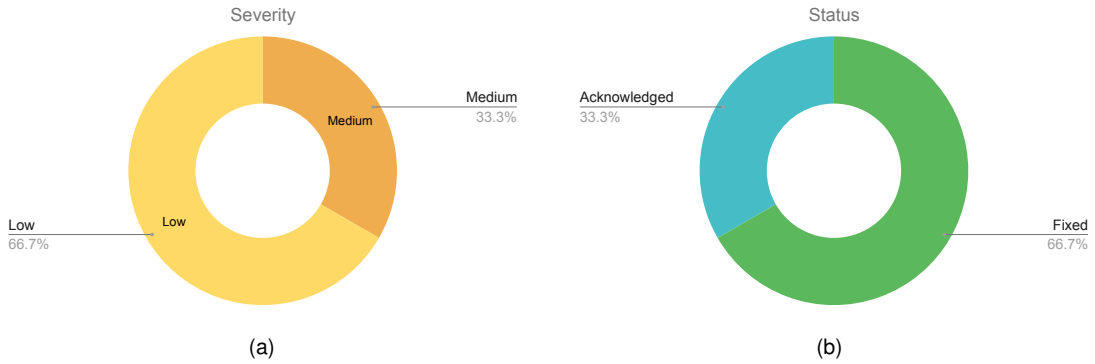

Fig. 1: Distribution of issues: Critical (0), **High** (0), **Medium** (1), **Low** (2), **Undetermined** (0), **Informational** (0), **Best Practices** (0).
Distribution of status: Fixed (2), **Acknowledged** (1), **Mitigated** (0), **Unresolved** (0)

Summary of the Audit

Audit Type	Security Review
Final Report	February 18, 2026
Initial Commit	`30eff1a`
Final Commit	`824e6bb`
Documentation Assessment	High
Test Suite Assessment	High

2 Audited Files

	Contract	LoC	Comments	Ratio	Blank	Total
1	src/OprfKeyGen.sol	114	53	46.5%	16	183
2	src/BabyJubJub.sol	456	145	31.8%	32	633
3	src/OprfKeyRegistry.sol	552	257	46.6%	81	890
	Total	**1122**	**455**	**40.6%**	**129**	**1706**

3 Summary of Issues

	Finding	Severity	Update
1	participants can reuse past rounds contributions from other parties	Medium	Fixed
2	Deleted `oprfKeyId` can be initiated again	Low	Fixed
3	`numPeers` should be exact	Low	Acknowledged

4 System Overview

The **OprfKeyRegistry** contract coordinates a multi-round MPC protocol among a fixed set of registered OPRF peers to generate, reshare, and register an OPRF public key (a BabyJubJub point) under a given OPRF key identifier (`oprfKeyId`). The contract is deployed behind an upgradeable proxy and initialized by the owner, who sets the Groth16 verifier address, the `(threshold, numPeers)` parameters, and an initial keygen admin. After the owner registers exactly `numPeers` distinct peer addresses (each assigned a stable `partyId`), the system is marked ready and key-generation sessions may begin.

Protocol state for each `oprfKeyId` is stored in `runningKeyGens[oprfKeyId]` (an `OprfKeyGen.OprfKeyGenState`), while finalized keys are stored in `oprfKeyRegistry[oprfKeyId]` as `RegisteredOprfPublicKey {key,epoch}`. A session can be started either as a fresh key generation (`initKeyGen`) or as a reshare (`initReshare`) for an existing key, which increments the epoch and re-randomizes / redistributes shares while preserving the registered public key identity.

Following the three-round structure used by the protocol, round progression is driven by peer submissions and contract-emitted events:

a. Round 1: Each registered peer submits a `Round1Contribution` containing a commitment share (`commShare`), a commitment to polynomial coefficients (`commCoeffs`), and an ephemeral BabyJubJub public key (`ephPubKey`).
   - In key-gen all peers are treated as PRODUCERS and must provide non-empty commitments; the contract aggregates `commShare` values into `keyAggregate`.
   - In reshare, peers may act as PRODUCER or CONSUMER. Producers must match their previous share commitment (prevShareCommitments) to prove consistent input, and once threshold producers are identified the contract computes and stores Lagrange coefficients for round 2.
   - When all peers have submitted round 1, the contract transitions to Round 2 (or marks the session STUCK if too few producers volunteered in a reshare) and emits SecretGenRound2 (key-gen) or the corresponding reshare round event.

b. Round 2: Only PRODUCERS submit a `Round2Contribution`, which contains encrypted share material for recipients (`SecretGenCiphertext[] ciphers`) and a compressed Groth16 proof (`compressedProof`). The contract:
   (a) enforces correct round and one-submission-per-party
   (b) validates BabyJubJub points in ciphertext commitments
   (c) accumulates shareCommitments (plain sum for key-gen; Lagrange-weighted accumulation for reshare)
   (d) verifies the Groth16 proof via a configured verifier (currently hard-coded public input layouts for (numPeers,threshold) = $(3,2)$ and $(5,3)$).

   Once all required producers have submitted (all peers for key-gen; `threshold` for reshare), the contract enters Round 3 and emits `SecretGenRound3` (key-gen) or `ReshareRound3` (reshare, also publishing Lagrange coefficients and epoch).

c. Round 3: All peers submit a final "done" acknowledgment (addRound3Contribution). When all parties have acknowledged, the contract finalizes:
   - For key-gen (epoch $== 0$), it registers `keyAggregate` as the OPRF public key for `oprfKeyId`.
   - For reshare, it updates only the stored epoch. In both cases it persists prevShareCommitments = shareCommitments for future reshares, emits `SecretGenFinalize`, and resets transient session state while keeping the key reusable.

The accompanying **OprfKeyGen.sol** file is a library that defines the protocol's shared types (round enums, role enums, contribution structs, ciphertext format, and the per-key `OprfKeyGenState`) plus helper routines to initialize, reset, and delete protocol state safely across key-gen and reshare sessions.

The **BabyJubJub contract** supplies the elliptic-curve arithmetic and validation used throughout (point addition, scalar multiplication, on-curve / subgroup checks, and Lagrange coefficient computation), ensuring deterministic, field-safe computations for all curve-based commitments and aggregates.

4.1 Access Control

Separates governance from participation. The owner registers the peer set and is the only party allowed to authorize upgrades (UUPS). Keygen admins (managed by `addKeyGenAdmin` and `revokeKeyGenAdmin`) are the only entities allowed to start, abort, delete keys, or trigger (re)initialization flows. Only registered peers may submit round contributions; submissions are enforced via strict `msg.sender → partyId` checks, round guards (`WrongRound`), and replay protection (`AlreadySubmitted`). Readiness is enforced by `isReady`, and proxy/initializer safety is enforced via `onlyProxy` and `onlyInitialized`.

5 Risk Rating Methodology

The risk rating methodology used by Nethermind Security follows the principles established by the OWASP Foundation. The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;

b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;

c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;

b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;

c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	**High**	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	**Medium**	**High**
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, Nethermind Security also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;

b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;

c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [Medium] participants can reuse past rounds contributions from other parties

File(s): src/OprfKeyRegistry.sol

Description: The key generation process consists of three rounds in which all registered peers must participate. Round 1 requires submitting secret commitments and aggregating the key, Round 2 involves submitting zero-knowledge proofs of correct secret calculation, and Round 3 requires acknowledging the reception of ciphertexts.

The issue arises from the fact that nothing prevents a participant from submitting another party's round contribution as their own. Round 1 contributions only verify the validity of the commitment and ensure that the participant has not already contributed:

```
function addRound1KeyGenContribution(uint160 oprfKeyId, OprfKeyGen.Round1Contribution calldata data)
    public virtual onlyProxy isReady
{
    // return the partyId if sender is really a participant
    uint16 partyId = _internParticipantCheck();
    _curveChecks(data.commShare);
    if (data.commCoeffs == 0) revert BadContribution();
    OprfKeyGen.OprfKeyGenState storage st = _addRound1Contribution(oprfKeyId, partyId, data);
    // check that this is a key-gen
    if (st.generatedEpoch != 0) {
        revert BadContribution();
    }
    st.nodeRoles[msg.sender] = OprfKeyGen.KeyGenRole.PRODUCER;
    st.numProducers += 1;
    _addToAggregate(st.keyAggregate, data.commShare);
    _tryEmitRound2Event(oprfKeyId, numPeers, st);
        emit OprfKeyGen.KeyGenConfirmation(oprfKeyId, partyId, 1, st.generatedEpoch);
}
```

Round 2 contributions have similar checks, with the addition of zero-knowledge proof verification that embeds public data observable by other participants:

```
function addRound2Contribution(uint160 oprfKeyId, OprfKeyGen.Round2Contribution calldata data)
    public virtual onlyProxy isReady
{
    if (data.ciphers.length != numPeers) revert BadContribution();
        OprfKeyGen.OprfKeyGenState storage st = runningKeyGens[oprfKeyId];
    if (st.currentRound != OprfKeyGen.Round.TWO) revert WrongRound(st.currentRound);

    uint16 partyId = _internParticipantCheck();
    if (st.round2Done[partyId]) revert AlreadySubmitted();
    if (OprfKeyGen.KeyGenRole.PRODUCER != st.nodeRoles[msg.sender]) revert BadContribution();
    // --SNIP
    if (numPeers == 3 && threshold == 2) {
        IVerifierKeyGen13 keyGenVerifier13 = IVerifierKeyGen13(keyGenVerifier);

        uint256[PUBLIC_INPUT_LENGTH_KEYGEN_13] memory publicInputs;

        BabyJubJub.Affine[] memory pubKeyList = _loadPeerPublicKeys(st);
        publicInputs[0] = pubKeyList[partyId].x;
        publicInputs[1] = pubKeyList[partyId].y;
        publicInputs[2] = st.round1[partyId].commShare.x;
        publicInputs[3] = st.round1[partyId].commShare.y;
        publicInputs[4] = st.round1[partyId].commCoeffs;
        publicInputs[5 + (numPeers * 3)] = threshold - 1;
        for (uint256 i = 0; i < numPeers; ++i) {
            publicInputs[5 + i] = data.ciphers[i].cipher;
                publicInputs[5 + numPeers + (i * 2) + 0] = data.ciphers[i]commitment.x;
            publicInputs[5 + numPeers + (i * 2) + 1] = data.ciphers[i].commitment.y;
            publicInputs[5 + (numPeers * 3) + 1 + (i * 2) + 0] = pubKeyList[i].x;
            publicInputs[5 + (numPeers * 3) + 1 + (i * 2) + 1] = pubKeyList[i].y;
            publicInputs[5 + (numPeers * 5) + 1 + i] = data.ciphers[i].nonce;
        }
        keyGenVerifier13.verifyCompressedProof(data.compressedProof, publicInputs);
        }
    // --SNIP
```

```
}
```

As demonstrated above, neither function checks for duplicate contributions across different participants. This allows participants to replicate contributions from others, resulting in the final key being generated with fewer unique contributions than required. This vulnerability directly contradicts the specification outlined in the whitepaper this smart contract implements, specifically in *Figure 7: Proposal 2 for key generation using ZK proofs* says:

> The smart contract checks if $X_i = X_j$ for any $i \neq j$ in which case it aborts. Otherwise, it computes the public key from the commitments $pk = \prod_{i=1}^{t} X_i$.

Recommendation(s): Multiple approaches can address this issue. One solution, as suggested in the whitepaper, is to implement duplicate contribution detection. Alternatively, the `partyId` could be included as a public input in the zero-knowledge proofs to cryptographically bind each contribution to its participant.

Status: Fixed

Update from the client: Fixed in commit 824e6bb

6.2 [Low] Deleted `oprfKeyId` can be initiated again

File(s): `src/OprfKeyRegistry.sol`

Description: An admin can delete an existing `oprfKeyId` by calling `deleteOprfPublicKey`. Deleted OPRF keys are expected to remain permanently deleted, as both the `initKeyGen` and `initReshare` functions explicitly check against the `DELETED` state:

```
function initKeyGen(uint160 oprfKeyId) public virtual onlyProxy isReady onlyAdmin {
    if (oprfKeyId == 0) revert BadContribution();
    // Check that this oprfKeyId was not used already
    BabyJubJub.Affine storage publicKey = oprfKeyRegistry[oprfKeyId].key;
    OprfKeyGen.OprfKeyGenState storage st = runningKeyGens[oprfKeyId];

    // check if deleted
@==>    if (st.currentRound == OprfKeyGen.Round.DELETED) revert DeletedId(oprfKeyId);
    // -- SNIP
```

However, an admin can intentionally or unintentionally bypass this protection by calling `abortKeyGen` on a deleted `oprfKeyId`, which resets its state to `NOT_STARTED`, thereby allowing the key generation process to be reinitiated:

```
function abortKeyGen(uint160 oprfKeyId) public virtual onlyProxy isReady onlyAdmin {
    OprfKeyGen.OprfKeyGenState storage st = runningKeyGens[oprfKeyId];
    if (st.currentRound == OprfKeyGen.Round.NOT_STARTED) {
        revert UnknownId(oprfKeyId);
    }
    st.reset(numPeers, peerAddresses);
    emit OprfKeyGen.KeyGenAbort(oprfKeyId);
}

function reset(OprfKeyGenState storage st, uint256 numPeers, address[] memory peerAddresses) internal {
    _reset(st, numPeers, peerAddresses);
    st.currentRound = Round.NOT_STARTED;
}
```

Recommendation(s): Consider adding a validation check in the `abortKeyGen` function to prevent aborting operations on `oprfKeyId` entries that are in the `DELETED` state.

Status: Fixed

Update from the client: Fixed in commit 61f492c

6.3 [Low] numPeers should be exact

File(s): src/OprfKeyRegistry.sol

Description: The circuit using the number of peers requires it to be a fixed number but the initialiser allows for any value for the number of peers that is greater than 2.

The initialiser allows the passing of any uint16 value as threshold or numPeers seen below:

```
/// @param _threshold The threshold number of peers required for key generation.
/// @param _numPeers The number of peers participating in the key generation.
function initialize(
    address _owner,
    address _keygenAdmin,
    address _keyGenVerifierAddress,
    uint16 _threshold,
    uint16 _numPeers
) public virtual initializer {
    // -- SNIP
    // @audit this should have a fixed number of 2 or 3 so it does not revert in round 2 contribution
    threshold = _threshold;
    // @audit this should have a fixed number of 3 or 5
    numPeers = _numPeers;
    isContractReady = false;
}
```

When performing the ZK proof verification and building the public inputs, there is a check that numPeers and threshold are either (3,2) or (5,3); otherwise, the contract reverts:

```
if (numPeers == 3 && threshold == 2) {
    // -- SNIP
    keyGenVerifier13.verifyCompressedProof(data.compressedProof, publicInputs);
} else if (numPeers == 5 && threshold == 3) {
    // -- SNIP
    keyGenVerifier25.verifyCompressedProof(data.compressedProof, publicInputs);
} else {
    revert UnsupportedNumPeersThreshold();
}
```

This lack of a check allows the owner to incorrectly set either or both values, causing a Dos on all key generation.

Recommendation(s): Restrict peers and threshold to be the corresponding values required by the circuit.

Status: Acknowledged.

Update from the client: Acknowledged.

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- Testing documentation: Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about World's documentation

`World's` team provided an overview of the main system components during the kick-off call with a detailed explanation of the intended functionalities and use cases of the system. Moreover, the team addressed all questions and concerns raised by the Nethermind Security team, providing valuable insights and a comprehensive understanding of the project's technical aspects.

8 Test Suite Evaluation

8.1 Tests Output

```
> forge test

Ran 2 tests for test/Groth16Verifier.t.sol:KeyGen13Groth16Verifier
[PASS] testCompressedProof() (gas: 354600)
[PASS] testUncompressedProof() (gas: 345131)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 10.13ms (14.70ms CPU time)

Ran 12 tests for test/OprfKeyRegistry.keygen.t.sol:OprfKeyRegistryKeyGenTest
[PASS] testAbortBeforeRound2() (gas: 9951773)
[PASS] testAbortBeforeRound3() (gas: 14700890)
[PASS] testAbortDuringRound1() (gas: 9152137)
[PASS] testAbortDuringRound2() (gas: 11418263)
[PASS] testAbortDuringRound3() (gas: 14710451)
[PASS] testAbortKeyGenBeforeRound1() (gas: 7622333)
[PASS] testDeleteAfterInitKeyGenShouldFail() (gas: 7573560)
[PASS] testDeleteBeforeRound1() (gas: 7573252)
[PASS] testDeleteBeforeRound2() (gas: 7573428)
[PASS] testDeleteBeforeRound3() (gas: 7573307)
[PASS] testKeyGen() (gas: 7569438)
[PASS] testKeyGenThenDelete() (gas: 7431631)
Suite result: ok. 12 passed; 0 failed; 0 skipped; finished in 318.16ms (996.41ms CPU time)

Ran 11 tests for test/BabyJubjub.t.sol:BabyJubJubTest
[PASS] testAddGeneratorToItself() (gas: 62960)
[PASS] testAddIdentity() (gas: 6519)
[PASS] testCurveChecks() (gas: 1427037)
[PASS] testGeneratorOnCurve() (gas: 362565)
[PASS] testIdentityPoint() (gas: 352725)
[PASS] testIsEmpty() (gas: 7700)
[PASS] testIsEqual() (gas: 10897)
[PASS] testLagrangeCoeffsDegree2() (gas: 406230)
[PASS] testLagrangeCoeffsDegree3() (gas: 406527)
[PASS] testScalarMul() (gas: 1732944)
[PASS] testThreeTimes() (gas: 834692)
Suite result: ok. 11 passed; 0 failed; 0 skipped; finished in 318.22ms (33.65ms CPU time)

Ran 14 tests for test/OprfKeyRegistry.admin.t.sol:OprfKeyRegistryTest
[PASS] testConstructedCorrectly() (gas: 4530609)
[PASS] testDeleteBeforeKeyGen() (gas: 31526)
[PASS] testInitKeyGenResubmit() (gas: 419786)
[PASS] testInitKeyGenRevokeRegisterAdmin() (gas: 463457)
[PASS] testInitKeyGenWithZero() (gas: 21754)
[PASS] testInitReshareBeforeKeyGen() (gas: 30767)
[PASS] testRegisterAdminTwice() (gas: 28241)
[PASS] testRegisterParticipants() (gas: 4730879)
[PASS] testRegisterParticipantsNotDistinct() (gas: 4533694)
[PASS] testRegisterParticipantsNotTACEO() (gas: 4534528)
[PASS] testRegisterParticipantsWrongNumberKeys() (gas: 4532058)
[PASS] testRevokeAdminThatIsNoAdmin() (gas: 30516)
[PASS] testRevokeLastAdmin() (gas: 32362)
[PASS] testUpdateParticipants() (gas: 112258)
Suite result: ok. 14 passed; 0 failed; 0 skipped; finished in 318.28ms (40.39ms CPU time)

Ran 2 tests for test/OprfKeyRegistryUpgrade.t.sol:OprfKeyRegistryUpgradeTest
[PASS] testOwnershipTransfer() (gas: 42680)
[PASS] testUpgrade() (gas: 19473925)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 388.28ms (284.15ms CPU time)

Ran 32 tests for test/OprfKeyRegistry.reshare.t.sol:OprfKeyRegistryReshareTest
[PASS] testAbortAfterKeyGenThenReshare() (gas: 14827211)
[PASS] testAbortAfterReshareThenReshare() (gas: 21641715)
[PASS] testAbortBeforeRound2() (gas: 9954940)
[PASS] testAbortBeforeRound3() (gas: 14704703)
[PASS] testAbortDuringKeyGenAndAddRound2Contribution() (gas: 28404036)
[PASS] testAbortDuringKeyGenAndAddRound3Contribution() (gas: 28399644)
[PASS] testAbortDuringKeyGenInitAgainAndAddRound2Contribution() (gas: 7590266)
[PASS] testAbortDuringKeyGenInitAgainAndAddRound3Contribution() (gas: 7586210)
```

```
[PASS] testAbortDuringKeyGenMultipleTimes() (gas: 7377779)
[PASS] testAbortDuringReshareMultipleTimes() (gas: 16044260)
[PASS] testAbortDuringRound1() (gas: 9155094)
[PASS] testAbortDuringRound2() (gas: 11421780)
[PASS] testAbortDuringRound3() (gas: 14714400)
[PASS] testAbortKeyGenBeforeRound1() (gas: 7624986)
[PASS] testAbortReshareBeforeRound1() (gas: 14498728)
[PASS] testAbortReshareBeforeRound2() (gas: 16421492)
[PASS] testAbortReshareBeforeRound3() (gas: 21193187)
[PASS] testAbortReshareDuringRound1() (gas: 16039044)
[PASS] testAbortReshareDuringRound2() (gas: 18196452)
[PASS] testAbortReshareDuringRound3() (gas: 21212935)
[PASS] testDeleteAfterInitKeyGenShouldFail() (gas: 7576177)
[PASS] testDeleteBeforeRound1() (gas: 7575693)
[PASS] testDeleteBeforeRound2() (gas: 7575913)
[PASS] testDeleteBeforeRound3() (gas: 7575770)
[PASS] testInitReshareResubmit() (gas: 7813021)
[PASS] testKeyGen() (gas: 7571699)
[PASS] testKeyGenThenDelete() (gas: 7434077)
[PASS] testReshare1() (gas: 14448144)
[PASS] testReshare1ThenDelete() (gas: 14237236)
[PASS] testReshare2() (gas: 21262829)
[PASS] testReshare2ThenDelete() (gas: 21051649)
[PASS] testReshareIsStuck() (gas: 22813306)
Suite result: ok. 32 passed; 0 failed; 0 skipped; finished in 388.48ms (4.23s CPU time)

Ran 6 test suites in 391.28ms (1.74s CPU time): 73 tests passed, 0 failed, 0 skipped (73 total tests)
```

8.2 Automated Tools

8.2.1 AuditAgent

The AuditAgent is an AI-powered smart contract auditing tool that analyses code, detects vulnerabilities, and provides actionable fixes. It accelerates the security analysis process, complementing human expertise with advanced AI models to deliver efficient and comprehensive smart contract audits. Available at https://app.auditagent.nethermind.io.

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;

- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to Nethermind in order that Nethermind could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. Nethermind has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, Nethermind disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. Nethermind does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and Nethermind will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.